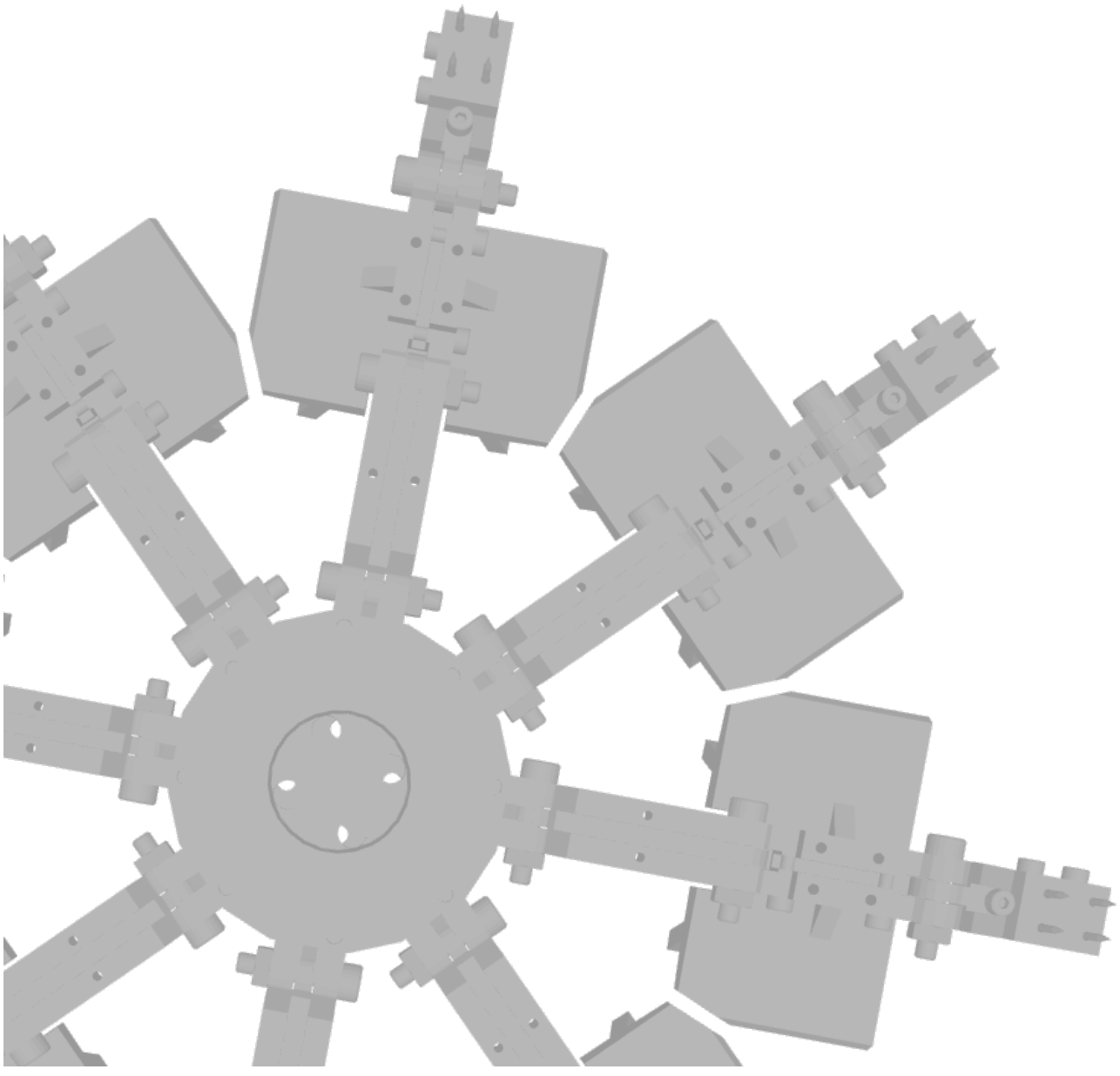


# 2 | Modeling and Control of Limbed Robots

The former chapter describes common locomotion systems used for field robots, including new hybrid trends. In addition, the objectives of this research have been introduced, highlighting the importance of limbed locomotion for the system under analysis. This chapter introduces the main theory behind limbed robot modeling and control, which are fundamental concepts for motion control and performance analysis.

First, the difference between limbed and legged robots is formally explained. Once this difference is clear, the kinematic model is introduced, describing forward, inverse, and differential kinematics, and why they are important for any motion task. Then, the fundamental equations of the dynamic model are presented, along with a simpler approach to describing them. Finally, a common control architecture is described, with an overview of classical and novel controller algorithms. Those concepts are described for legged robots and manipulators, which are the basics for modeling and controlling limbed robots.

The tools to describe and analyze performance are presented in chapter 3.

## 2.1. Limbed vs Legged Robot

In anatomy, it is clear the difference between leg, arm, and limb, this difference can be defined in the same way in robotics. "Limb" encompasses both leg and arm, the former has locomotion as a main function, and the latter ensures object manipulation and complex interactions with the environment.

Theory on manipulators (robotic arms) dates back to the 50s, with the first example Unimate [33]. In the next years, also legged robots were coming, with pioners McGhee and Frank [99]. The first definitions of limb mechanism in robotics can be found in Koyachi's works. [78][4][145].

First, an important classification has to be done. Robots can be classified as those with a fixed base, **robot manipulators**, and those with a mobile base, **mobile robots**[134] To introduce legged robots, a definition of manipulator and legged robot is needed. In the 80s, the Robotics Institute of America (RIA) [134][70] defined manipulators as "robot":
**Definition 2.1.1.** *A **robot** is a reprogrammable multifunctional **manipulator** designed to move material, parts, tools, or specialized devices through variable programmed motions for the performance of a variety of tasks.*

Whenever the base can perform some motion, it is considered a **mobile manipulator** (see Fig.2.1), entering the class of **mobile robots**.



(a) CHIMERA Mobile manipulator. (b) KUKA-DLR manipulator.

Figure 2.1: [35]Mobile manipulator and [11]robotic arm

Raibert[115] and Todd[150] give an extensive analysis of walking machines, while a clear definition of a legged robot can be found in [134] or [132]:
**Definition 2.1.2.** *[134]**Legged Robots** are made of multiple rigid bodies, interconnected by joints. Some of these bodies form lower limbs, whose extremities (feet) periodically come in contact with the ground to realize locomotion*
**Definition 2.1.3.** *[132]**Legged Robots** use legs to move from one place to another. Legs provide an active suspension, so the motion of the main body of the robot can be largely decoupled from the terrain profile. With each step, a leg is temporarily lifted off the ground, so that discontinuous terrain can be overcome as well, allowing locomotion in places out*

*of reach otherwise.*

In the same article[132], legs in robotics are defined as:
**Definition 2.1.4.** ***Legs*** *are usually articulated rigid bodies, assumed to contact the environment only with their end effector(EE). In most cases, this contact is unilateral, meaning that the robot can push but not pull on contact surfaces*

In general, the design of legs is done to achieve walking locomotion (such as the quadrupeds in Fig.1.4). Using proper mechanical structures or innovative solutions[137][113][19], also locomotion in walls and ceilings is possible, obtaining **climbing robots** (as the one in Fig.1.5a).

Finally, from Koyachi's works on multi-limbed robots, an explicit definition of limbed mechanism is given [78][4][145]:

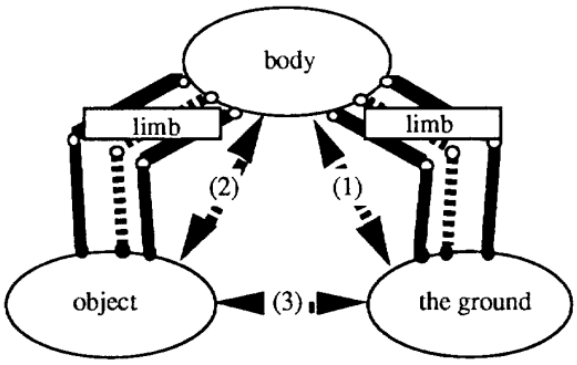



Figure 2.2: Scheme of Limb mechanism.[78]

**Definition 2.1.5.** *In robotics, a* ***Limb mechanism*** *is a link that connects the robot body with the environment. The objective with limb mechanism is to obtain the following motions (as in Fig.2.2) by moving each limb appropriately:*

- *(1) Relative motion between body and ground that causes locomotion of the robot.*
- *(2) Relative action between an object and a body that will be used for handling and machining a small object.*
- *(3) Relative action between an object and base that will be used for transferring and handling of an object, machining of a large object.*

From another publication of the same author[145]:
**Definition 2.1.6.** *In robotics, a* ***Limb mechanism*** *is a linkage integrating both locomotion and manipulation functions. Ensuring the following features:*

- *There is no clear boundary between walking and working spaces, and total efficiency increases because limbs can be used for walking or working according to the situation*
- *Force generated by the hand increases and working space is expanded because limbs can be coordinated and support limbs can move the body*
- *This mechanism can be compact and light because two functions are integrated into one linkage.*

The working space, also called **workspace**, comes from manipulator theory, and it is defined as [134]:
**Definition 2.1.7.** *The **workspace** is the region described by the origin of the end-effector frame when all the manipulator joints execute all possible motions.*

A formal definition of multi-limb robot cannot be found in literature, but an intuitive one can be deducted from the ones above:
**Definition 2.1.8.** *A **multi-limb robot** is composed of a body and multiple limb mechanisms. Limb-end integrates both locomotion and manipulation through suction cups, magnets, adhesive materials, or miniature spine arrays [132][65][2][109]. In some cases, even **limb/leg**-wheeled [12][180][117][39][146] or multimodal mechanisms can be integrated[153][142]. If a transformable multimodal mechanism is used as limb-end, the robot can be defined **multi-modal**.*

SCAR-E in Fig.1.6 is a good example of a multi-limb robot, with its gripper(Fig.1.6c) designed for climbing, manipulating, and walking.

To avoid any confusion, a final comment on robotic system classification has to be done, regarding the class of **Humanoid robots**[110]. These robots have a human-like physical appearance, with both manipulator and walking abilities, but the mechanical structures (arms and legs) used to achieve those tasks are different. Multi-limbed robots instead perform multiple tasks with the same limb structure.

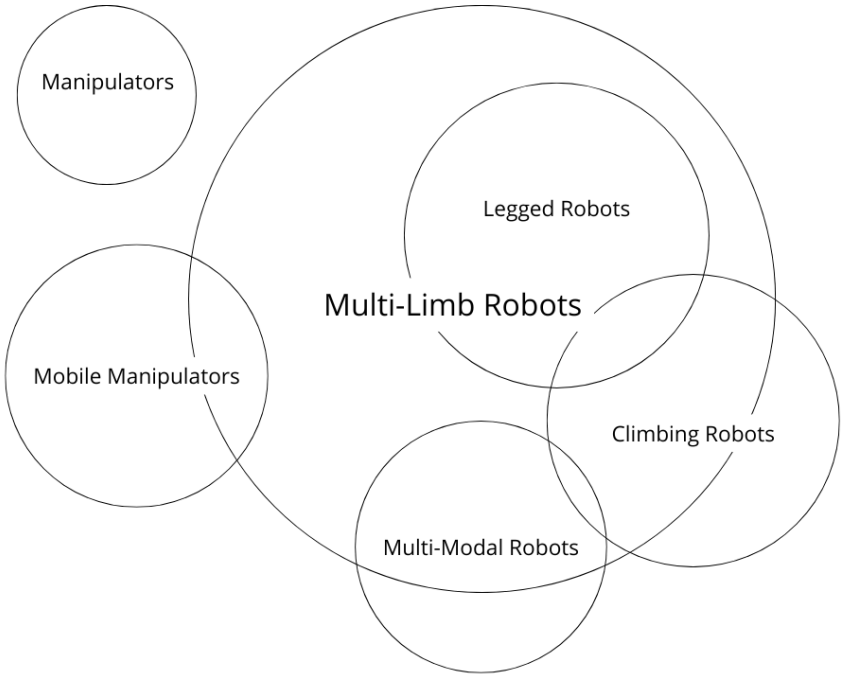



Figure 2.3: Diagram of some robot classes

It is hard to identify a clear boundary among some of these classes of robots. Fig.2.3 provides a diagram to summarize the concepts described in this section. This broad but mild classification highlights the reason why modeling and controlling limbed robots is not a simple task. A unique established theory doesn't exist. In the next chapters, first, state-of-the-art models of robot manipulators and legged robots are formalized, and then, some insights on possible solutions for limbed robots are given.

## 2.2. Kinematic Model

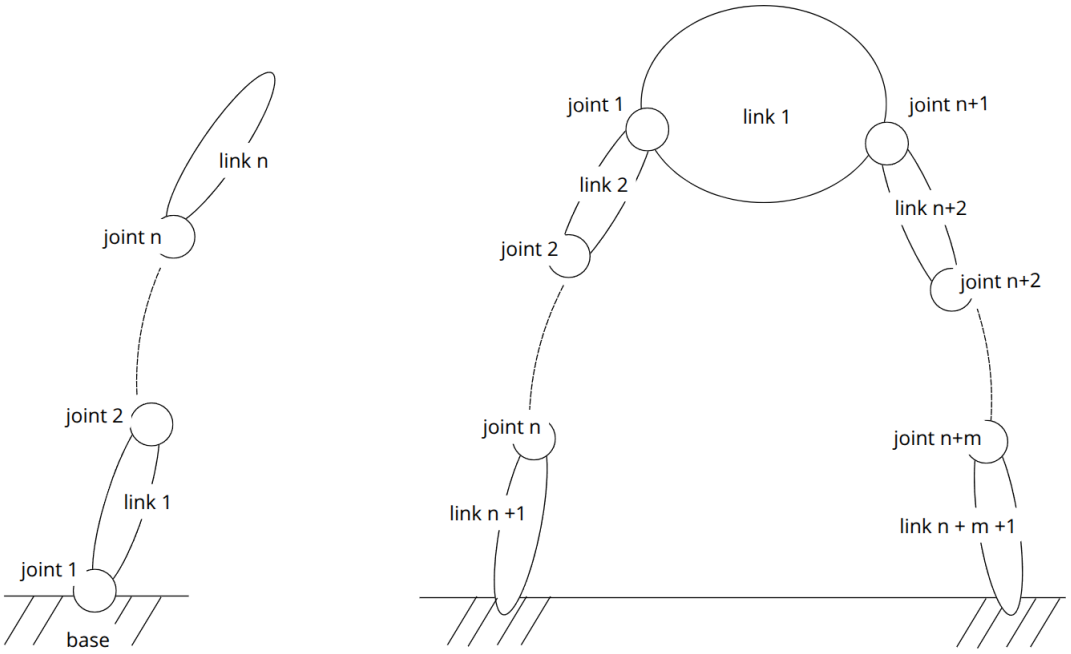
In mechanics, kinematics studies motion without considering its causes. When flexibility and softness are negligible in the system under analysis, rigid body theory is used.

The rigid parts of the robot, also called links, are connected by joints, usually prismatic or revolute [134]. The mobility of any robot is ensured by those joints, providing a single degree of freedom (DOF) each. An important concept when considering multi-body mechanics is the nature of the **kinematic chain**[134][3]. We classify it as:

- **open kinematic chain** when only one sequence of links connects the mechanical chain's two ends (Fig.2.4a). For example robot manipulators, but even dual-arm robots.

- **closed kinematic chain** when the sequence of links forms a loop(Fig.2.4b).

. As a Parallelogram mechanism (used in construction vehicles), or legged robots, considering the ”connection” between feet tip through a common ground [3][56].




(a) Open chain. (b) Closed chain.

Figure 2.4: Simple representation of open/closed kinematic chains.

In robotics, three main problems are considered: **forward**(FK), **inverse**(IK), and **differential kinematics** (DK). These are formally defined and solved for manipulators, legged, and multi-limb robots.

### 2.2.1. Forward Kinematic

The role of FK is to derive the relationship between joint variables and the pose of a final link frame[27][134]. The ”final link” depends on the type of robot. For a manipulator, it is the end effector, which is the last element of the open chain, usually carrying a tool. In legged and multi-limb robots, the pose of all legs/limbs tips is important.

Independently of the structure under analysis, the foundation of kinematics is the relationship between link frames. Each link is identified by an attached reference frame, and its pose is defined as the position and orientation of that frame [134][3][27]. The choice of

each link frame is not unique, the **Denavit-Hartenberg** (DH) convention [32] provides a standard way to define it.

Collecting all variables of the n joints in a vector $\mathbf{q} \in \mathbb{R}^n$ (time dependency is omitted), the final result of FK is a homogenous 4x4 transformation matrix $\mathbf{T_f^0}(\mathbf{q})$, "0" identify the inertia frame, "f" the final frame in consideration. This matrix contains information about the position and orientation:

$$\mathbf{T_f^0}(\mathbf{q}) = \begin{bmatrix} \mathbf{R_f^0}(\mathbf{q}) & {}^{\mathbf{0}}\mathbf{p_f}(\mathbf{q}) \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{2.1}$$

Here, the dependency on joint configuration $\mathbf{q}$ is explicit, to clarify how FK is a function of joint configuration, for notation conciseness, sometimes it is omitted. Furthermore, "0" is omitted in some cases, considering the inertial frame granted, $\mathbf{R_f^0} = \mathbf{R_f}$, ${}^{\mathbf{0}}\mathbf{p_f} = \mathbf{p_f}$, and so on. $\mathbf{T_f^0}$ is a powerful mathematical tool that not only contains information about the pose but also a matrix transformation to change the reference frame. If we consider the homogeneous coordinates ($\tilde{p} = \begin{bmatrix} \mathbf{p} & 1 \end{bmatrix}^T$) of a generic point A in space, it holds:

$${}^{\mathbf{0}}\tilde{\mathbf{p}}_{\mathbf{A}} = \mathbf{T_f^0} \cdot {}^{\mathbf{f}}\tilde{\mathbf{p}}_{\mathbf{A}} \tag{2.2}$$

Additionally, transformation matrix composition is as easy as a matrix product, and it can be chained.

$$\mathbf{T_f^0} = \mathbf{T_i^0} \cdot \mathbf{T_j^i} \cdot ... \cdot \mathbf{T_f^m} \tag{2.3}$$

The drawback of using homogeneous transformation is the redundancy of $\mathbf{R_f^0}$ as orientation description. Minimal or quai-minimal representations such as Euler angles or quaternion are often used [134].

In general, a pose in 3D space is described by 6 DOFs, 3 for rotation and 3 for orientation. In some cases, the task of the robotic structure requires less than 6 DOFs, and a task vector $\mathbf{x} \in \mathbb{R}^m$ is defined. The space in which the task is defined is called **operational/task space** (of dimension m), and it is always possible to derive a non-linear function, relating joint variables (in the so-called **joint space**) to this task vector:

$$\mathbf{x} = \mathbf{k}(\mathbf{q}) \tag{2.4}$$

$\mathbf{x}$ collects position ($\mathbf{p}$) and orientation ($\phi$) as: $\mathbf{x} = \begin{bmatrix} \mathbf{p} & \phi \end{bmatrix}^T$

A common minimal representation of orientation is the XYZ Euler angles also called Roll $\vartheta$, Pitch $\varphi$, Yaw $\psi$(RPY) (see Fig.2.5): $\phi = \begin{bmatrix} \vartheta & \varphi & \psi \end{bmatrix}^T$

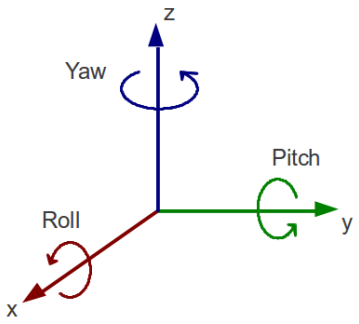



Figure 2.5: Roll Pitch Yaw angles

The explicit expression of the vectorial function $\mathbf{k}(\mathbf{q}) \in \mathbb{R}^m$ is often derived by geometrical analysis of the robotic structure ($\mathbf{T_f^0}$ can also be computed in this way).

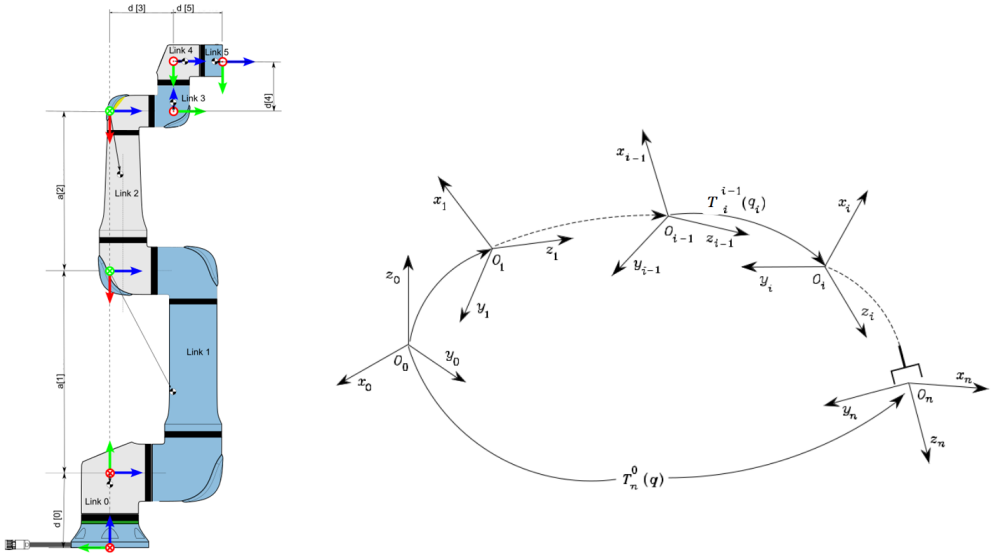



Figure 2.6: Block scheme of FK problem

Fig. 2.6 summarizes the kinematic problem and its resolution, Solving it is easy for an open kinematic chain robot, problems arise when the system presents closed chains.

The next part of this section presents techniques to solve FK for the robotics structure of interest.

## Manipulators and Single-Leg/Limb




(a) [126]UR DH frames. (b) [134]Open kinematic chain frames.

Figure 2.7: Example of DH Frames in UR series manipulator(a), and scheme of open kinematic chain frames(b)

In the context of **Robotic Manipulators**, the final frame (previously called **f**) is usually the final link frame (where a tool can be attached), defined as frame **n** (other times as end-effector frame **e**). Considering relation (2.3), for an open chain of n links, as the one in Fig.2.7b, the resolution of FK with **homogeneous transformations** is straightforward:

$$\mathbf{T_n^0(q) = T_1^0(q_1) \cdot T_2^1(q_2) \cdot \cdots \cdot T_i^{i-1}(q_i) \cdot \cdots \cdot T_n^{n-1}(q_n)} \tag{2.5}$$

With $q_i$ defining the joint connecting links $i-1$ to $i$, It is straightforward that $\mathbf{T_i^{i-1}(q) = T_i^{i-1}(q_i)}$.

In principle, the choice of each link frame is arbitrary, nevertheless, respecting the DH convention (as in Fig.2.7a) brings some advantages:

- It is a standard convention, which avoids any ambiguity.
- $\mathbf{T_i^{i-1}(q_i)}$ $\forall i$ can be computed in a standard form from DH Parameters.

In simple cases, such as the manipulator in Fig.2.8, the pose of the final frame can be computed explicitly using **geometric considerations**.

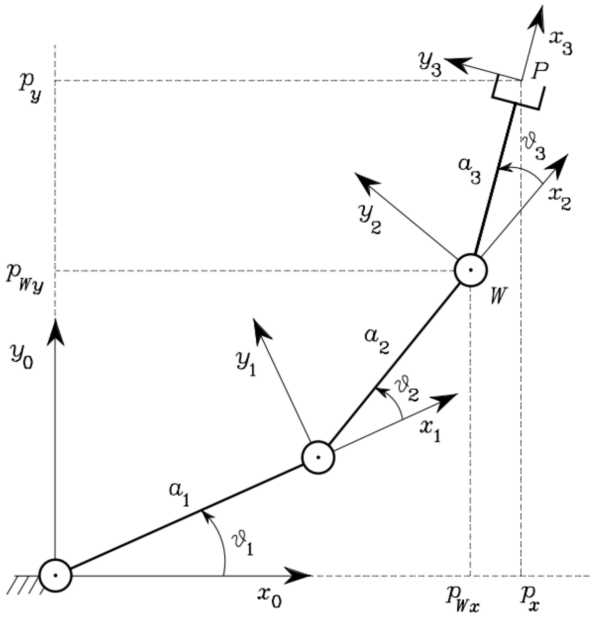



Figure 2.8: [134]Three-link planar arm

The Three-link planar arm above provides useful insights into the meaning of **task space**. This mechanical structure allows planar motion in the XY plane. with reference to the symbols in the figure, $\mathbf{q} = \begin{bmatrix} \vartheta_1 & \vartheta_2 & \vartheta_3 \end{bmatrix}^T$, $\mathbf{x} = \begin{bmatrix} p_x & p_y & \psi \end{bmatrix}^T$, where $p_x$, $p_y$ represents the EE position, while $\psi = \vartheta_1 + \vartheta_2 + \vartheta_3$ the EE orientation (as yaw angle). Deriving the expression of $\mathbf{T_3^0}(\mathbf{q})$ or $\mathbf{k}(\cdot) \in \mathbb{R}^3$ is not relevant here.

To introduce some commonly used reference frames, consider the following example. In Fig.2.9 below, the TITAN4 [138] is considered, with a fixed base. The inertial frame is the world frame **w**, **b** is a fixed frame attached to the base, frames from **0** to **n** identify robot links, and finally **e** is attached to the EE. Homogeneous transformation composition as in 2.5 is used:

$$\mathbf{T_e^w}(\mathbf{q}) = \mathbf{T_b^w} \cdot \mathbf{T_0^b} \cdot \mathbf{T_1^0}(\mathbf{q_1}) \cdot \cdots \cdot \mathbf{T_i^{i-1}}(\mathbf{q_i}) \cdot \cdots \cdot \mathbf{T_n^{n-1}}(\mathbf{q_n}) \cdot \mathbf{T_e^n} \tag{2.6}$$

Defining the pose of the EE frame in the world inertial frame.

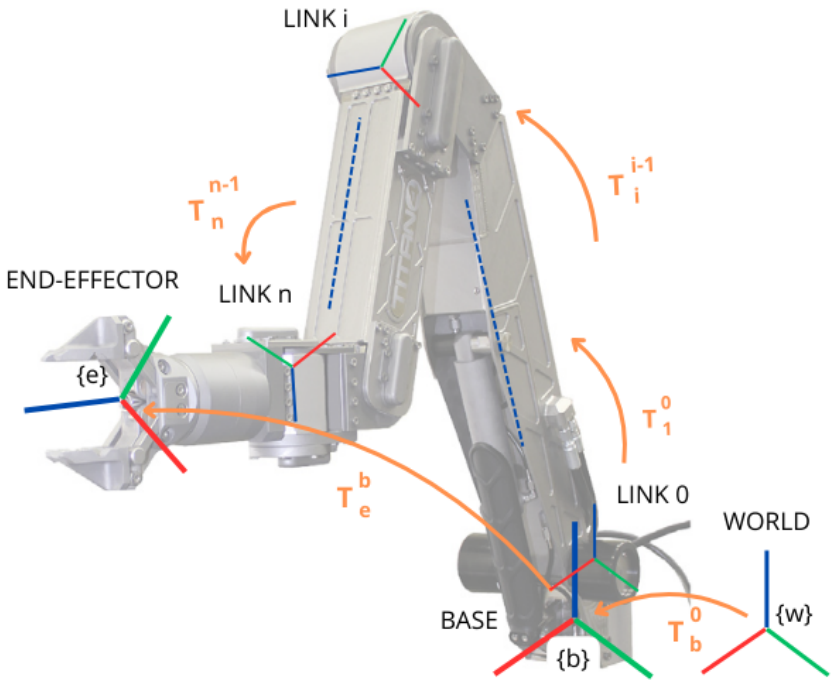
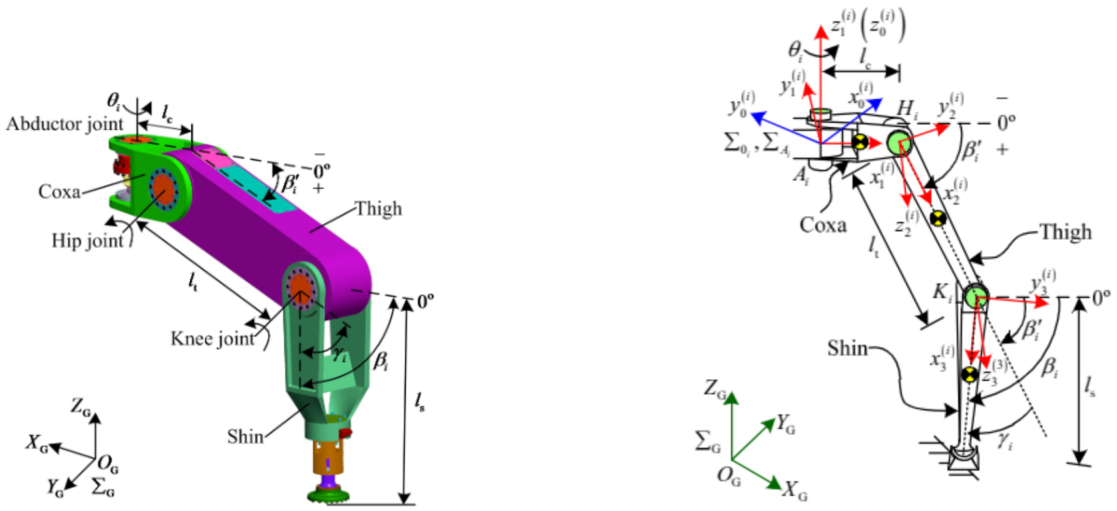



Figure 2.9: [138]Reference frames of fixed manipulator TITAN4

These concepts can be extended to any open-chain structure, even to a stand-alone robotic leg/limb (as the ones defined in Def.2.1.4 and Def.2.1.6).

For example, the robotic leg in Fig.2.10 has been described with the DH convention, to analyze the gait of a large-load-ratio six-legged robot [181]. Here, the model considers the open kinematic chain from the leg root to the leg tip (as the final frame). With reference to Fig.2.11, denoting the frame attached to the leg root as **r**, and the one on the leg tip as **f**, it is possible to derive $\mathbf{T_f^r}$. Then, $\mathbf{T_r^w}$ is needed to obtain position and orientation in the inertial frame. This transformation matrix can be derived as $\mathbf{T_r^w} = \mathbf{T_b^w} \cdot \mathbf{T_r^b}$, where $\mathbf{T_b^w}$ denote the transformation from world inertial frame to the robot base, while $\mathbf{T_r^b}$ from robot base to limb root. It is just a matter of transformation composition.

So, if the base pose is known ($\mathbf{T_b^w}$), it is possible to track the foot motion when the leg is above the ground. The same holds for any **limbs**, this modeling approach and classical manipulator FK theory can be used to plan manipulation tasks or limb motion.




(a) Concept of 3 DOF robotic leg for walking.

(b) DH frames for the same leg.

Figure 2.10: [181] Leg design and DH frames as open kinematic chain

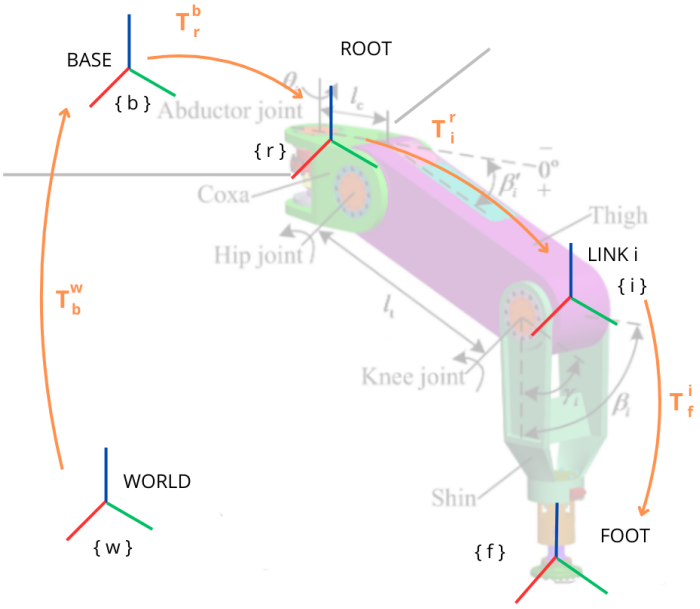



Figure 2.11: Reference frames of single leg

It is important to notice the difference between the fixed manipulator and leg/limb kinematic. While in the first $\mathbf{T_b^w}$ is fixed and usually known, in the latter, the base is in motion, so $\mathbf{T_b^w}$ changes over time. The same holds when a mobile manipulator kinematic is analyzed. In the next section, this difference is further analyzed to define the kinematic of legged robots.

## Legged Robots, Walking Machines

First, some terminology and symbols for walking robots is introduced.

- **Base**: mobile base of the legged/limbed robot, **body** and **torso** are synonyms.
- **Gait**: walking pattern
- **Gait cycle**: Period between two rising phases of the same leg during gait.
- **Stance phase**: phase in which the leg is in contact
- **Swing phase**: phase in which the leg is not in contact, changing the contact point

Expanding the notation used in the previous chapter to describe a single limb open chain model, it is possible to define all **N** legs kinematics. Denoting the limb index with $j = 1, .., N$ and the joint with $i$, $q_{j,i}$ identify joint $i$ of limb $j$. Usually, all legs have the same number of joints, but in general, $n_j$ is the number of joints of leg $j$, and $n = \sum_j n_j$ is the overall number of joints. $\mathbf{q_j} \in \mathbb{R}^{n_j}$ is the vector collecting all joint variables for leg $j$, while $\mathbf{q} \in \mathbb{R}^n$is the vector of all legs joint variables (system actuated DOFs). The relevant frame when analyzing the whole kinematic are **b**, **w**, $\mathbf{r_j}$, $\mathbf{f_j}$, $\mathbf{j_i}$ (denoting link $i$ of leg $j$). commonly, **b** is located in the base center of mass (CoM), $\mathbf{r_j}$ is rigidly attached to the base and identify the point where the first joint of the leg is attached. The definition of $\mathbf{j_i}$ instead is arbitrary, DH convention can be used, while $\mathbf{f_j}$ identify the foot tip, where the leg contacts the ground. Fig.2.12 gives a graphical representation of this model. A quadruped is represented, but the theory is generalizable for any N-legged robot.

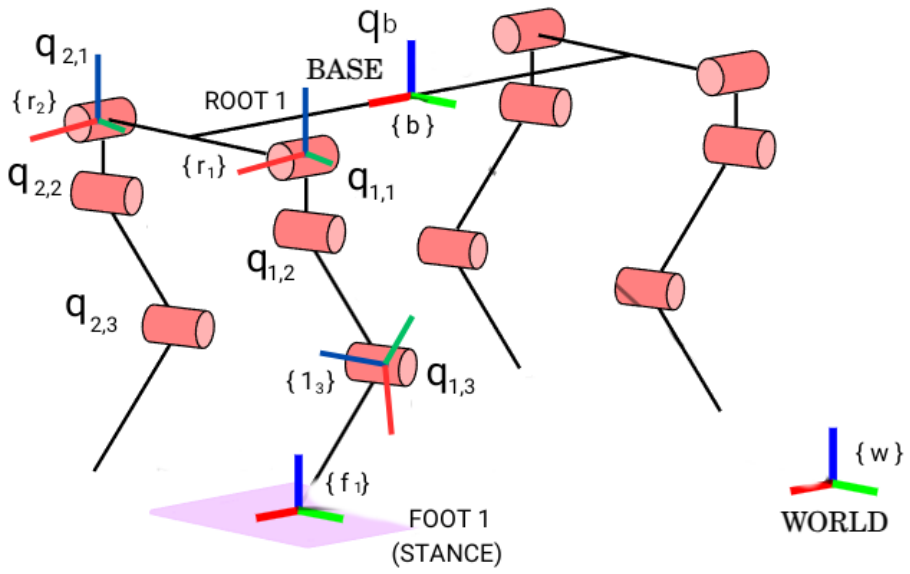



Figure 2.12: [104] Quadruped kinematic model

As introduced in the previous section, in legged robots the base moves in the world frame. In legged locomotion [56], there are two main problems:

1. Feet positioning to perform the gait
2. Base positioning to maintain stability

These two problems introduce two different kinematic objectives, discussed briefly here.

**Problem 1: Feet positioning**

Estimation of the base pose in the world is known as **localization** problem[149][49](present in any mobile robot), which is outside of the scope of this research. Furthermore, motion is achieved by a discrete placement of feet along the path. The problem of **path planning**[123] requires the definition of each foot stance position. This concept is discussed in 2.4.2, for now, it is important to anticipate that to achieve legged locomotion, foot position is usually computed in the base frame **b**.

For those reasons, this forward kinematic problem consists of deriving the pose of each foot tip $\mathbf{f_j}$ in the base frame **b** ($\mathbf{T^b_{f_j}}$) (sometimes in the world **w** frame,$\mathbf{T^w_{f_j}}$). Knowing the pose of the base **b** in the world **w** ($\mathbf{T^w_b}$), and all joint variables $\mathbf{q_j}$ of each leg j.

Under this hypothesis, the **floating base** model [97][174] is used. The base pose introduce 6 additional DOFs in 3D, $\mathbf{q_b} = \begin{bmatrix} x_b & y_b & z_b & \vartheta_b & \varphi_b & \psi_b \end{bmatrix}^T$ included in the system DOFs $\mathbf{q_{tot}} = \begin{bmatrix} \mathbf{q} & \mathbf{q_b} \end{bmatrix}^T \in \mathbb{R}^{n+6}$. $\mathbf{q_b}$ affect the system state but there isn't any dedicated actuator, so they are considered as unactuated virtual DOFs (see Fig.2.13).

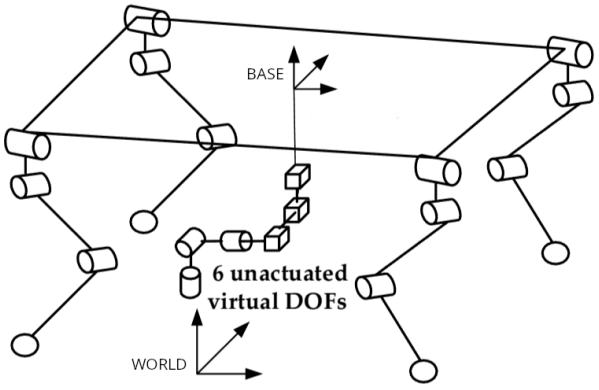



Figure 2.13: Floating base model

Refering to Fig.2.14, to derive $\mathbf{T^b_{f_j}}$ an equation similar to 2.6 is used:

$$\mathbf{T^b_{f_j}}(\mathbf{q_j}) = \mathbf{T^b_r} \cdot \mathbf{T^r_{j_0}} \cdot \mathbf{T^{j_0}_{j_1}}(\mathbf{q_{j,1}}) \cdot \cdots \cdot \mathbf{T^{j_{i-1}}_{j_i}}(\mathbf{q_{j,i}}) \cdot \cdots \cdot \mathbf{T^{j_{n_j-1}}_{j_{n_j}}}(\mathbf{q_{j,n_j}}) \cdot \mathbf{T^{j_{n_j}}_{f_j}}, \forall j = 1, ..., N \quad (2.7)$$

If instead the foot position in the world frame is needed, the floating base DOFs are used

$$\mathbf{T}^{\mathbf{w}}_{\mathbf{f_j}}(\mathbf{q_j}, \mathbf{q_b}) = \mathbf{T}^{\mathbf{w}}_{\mathbf{b}}(\mathbf{q_b}) \cdot \mathbf{T}^{\mathbf{b}}_{\mathbf{f_j}}(\mathbf{q_j}), \forall j = 1, ..., N \tag{2.8}$$

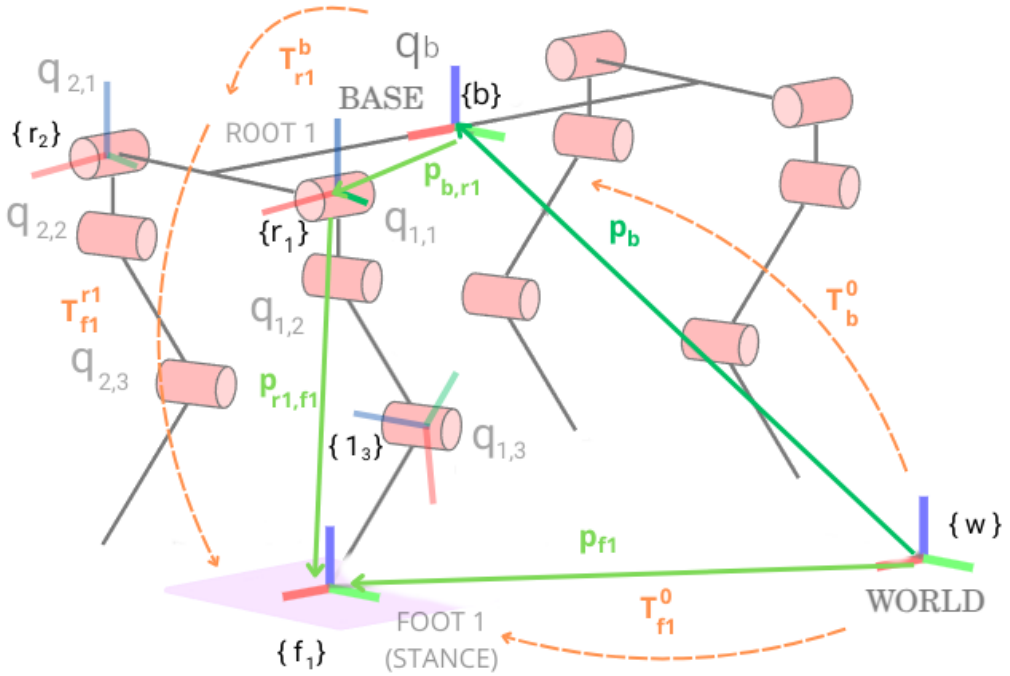



Figure 2.14: Floating base kinematic position vectors

**Problem 2: Base positioning**

In a legged machine, stability is a crucial aspect, explored in detail in 3. For now, take for granted that to maintain the base stable, the base CoM is positioned in a specific region from the stance feet contact points. For this reason, it is important to describe the pose of the base from each foot contact point frame $\mathbf{T}^{\mathbf{f_j}}_{\mathbf{b}}$.

This forward kinematic problem consists of the derivation of the base pose $\mathbf{b}$ respect each contact leg foot $\mathbf{f_j}$ ($\mathbf{T}^{\mathbf{f_j}}_{\mathbf{b}}, \forall j \in J_c$), given the vector of leg $j$ DOFs $\mathbf{q_j}$. Where $J_c$ defines the set of indices of legs in contact with the ground.

The kinematic model of each contact leg $j \in J_c$ is redefined for this purpose from the foot tip to the leg root as in Fig.2.15. Contact with the ground is assumed to take place such that the ground can exert only a "pushing"force on each leg but no moment. Thus, while we can model the contact between leg and ground as a spherical joint, care must be taken so that no pulls of the ground on the leg are required for a given gait[3] (The

above assumption excludes climbing and complex multi-limb robot structures).

In this model, each contact foot frame $\mathbf{f_j}$ is attached to the foot tip and considered aligned with the ground in the xy plane. The spherical joint models the ground interaction and is represented by 3 rotational DOFs $(q_{j,1}, q_{j,2}, q_{j,3})$ of virtual unactuated joint variables (as for the floating base). The derivation of $\mathbf{T_b^{f_j}}$ is just a matter of homogeneous transformation composition as already described in depth:

$$\mathbf{T_b^{f_j}(q_j)} = \mathbf{T_{j_1}^{f_j}(q_{j,1})} \cdot \mathbf{T_{j_2}^{j_1}(q_{j,2})} \cdot \cdot \mathbf{T_{j_i}^{j_{i-1}}(q_{j,i})} \cdot \cdots \mathbf{T_{j_{n_j}}^{j_{n_j-1}}(q_{j,n})} \cdot \mathbf{T_{r_j}^{j_{n_j}}} \cdot \mathbf{T_b^{r_j}}, \forall j \in J_c \qquad (2.9)$$

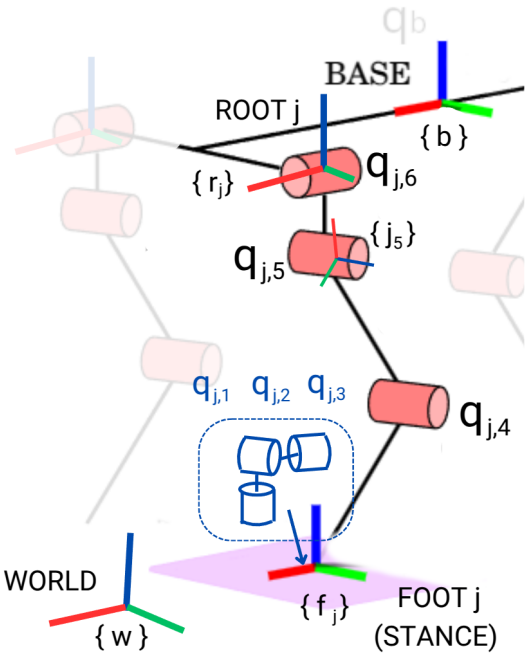



Figure 2.15: Kinematic model of stance leg from foot tip to leg root

## Limbed Robots, Multipurpose Machines

From some limbed robots described in chapter 1, the multifunctional aspect is clear. Those robots can move with walking locomotion, but also climbing, hybrid legged-wheeled driving, and even manipulate objects in the meantime[18][78][57]. As for the kinematic modeling of walking machines defined in the previous section, different problems have to be solved. In general, we are interested in:

- All limbs-end position respect base (or world)
- Base pose respect limb-end

The kinematic model for a generic multi-limb robot is as the one in Fig.2.16. The frames have the same meaning as for walking robots, except for the last frame referred to as $\mathbf{e_j}$. As for manipulators, any manipulation or locomotion "tool" can be attached to each limb's end frame (spine gripper, magnetic gripper, manufacturing tool, wheel, and so on...).

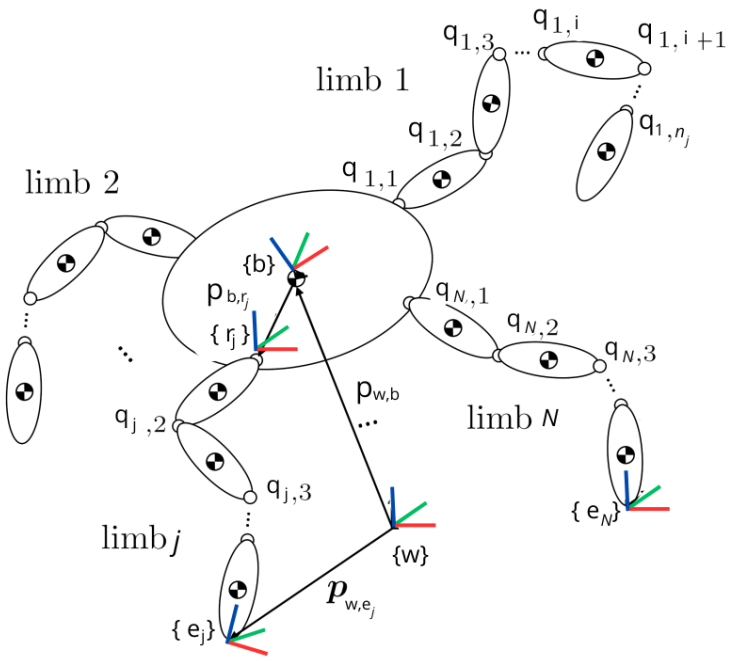



Figure 2.16: [122] model of multi-limb robot

To solve the two main forward kinematic problems listed above, the same resolution as in equation 2.9 (limb-end to base)and 2.8 (base to limb-end) can be applied.

Figure 2.17 below summarizes the different robots' FK problems described here.

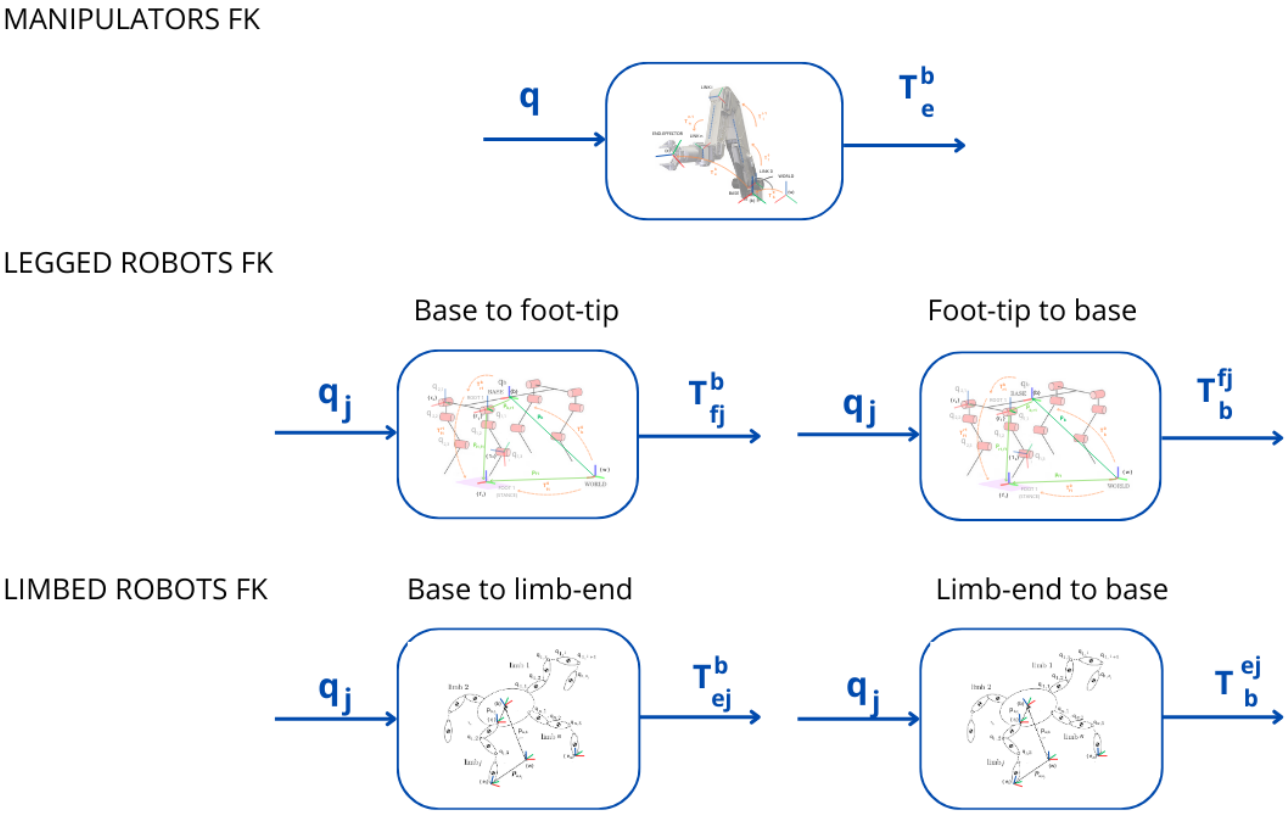



Figure 2.17: Scheme of FK problem for different robots

With a standard procedure, the homogeneous transformation matrix is a way to represent the "final" frame pose. From that matrix, it is possible to extract the explicit FK equation $k(\cdot)$ in the task space. Alternatively, geometric analysis is always possible, but there isn't a standard formulation procedure.

### 2.2.2. Inverse Kinematic

As the name suggests, IK solves the Inverse problem of FK, going from the final frame pose (or task space) to the joint space [134][3](as represented in Fig.2.18). From [134], in the context of **Manipulators**: *"The inverse kinematics problem consists of the determination of the joint variables corresponding to a given end-effector position and orientation. The solution to this problem is of fundamental importance to transform the motion specifications, assigned to the end-effector in the operational space, into the corresponding joint space motions that allow execution of the desired motion"*. So, in this formulation for dif-

ferent robotic structures, the ”end-effector” could be the mobile base of the legged/limbed robot or the limb/leg end, depending on the task to execute.




Figure 2.18: Scheme of general IK problems

Figure 2.19 tries to summarize the IK problem for different robots with a simple scheme.

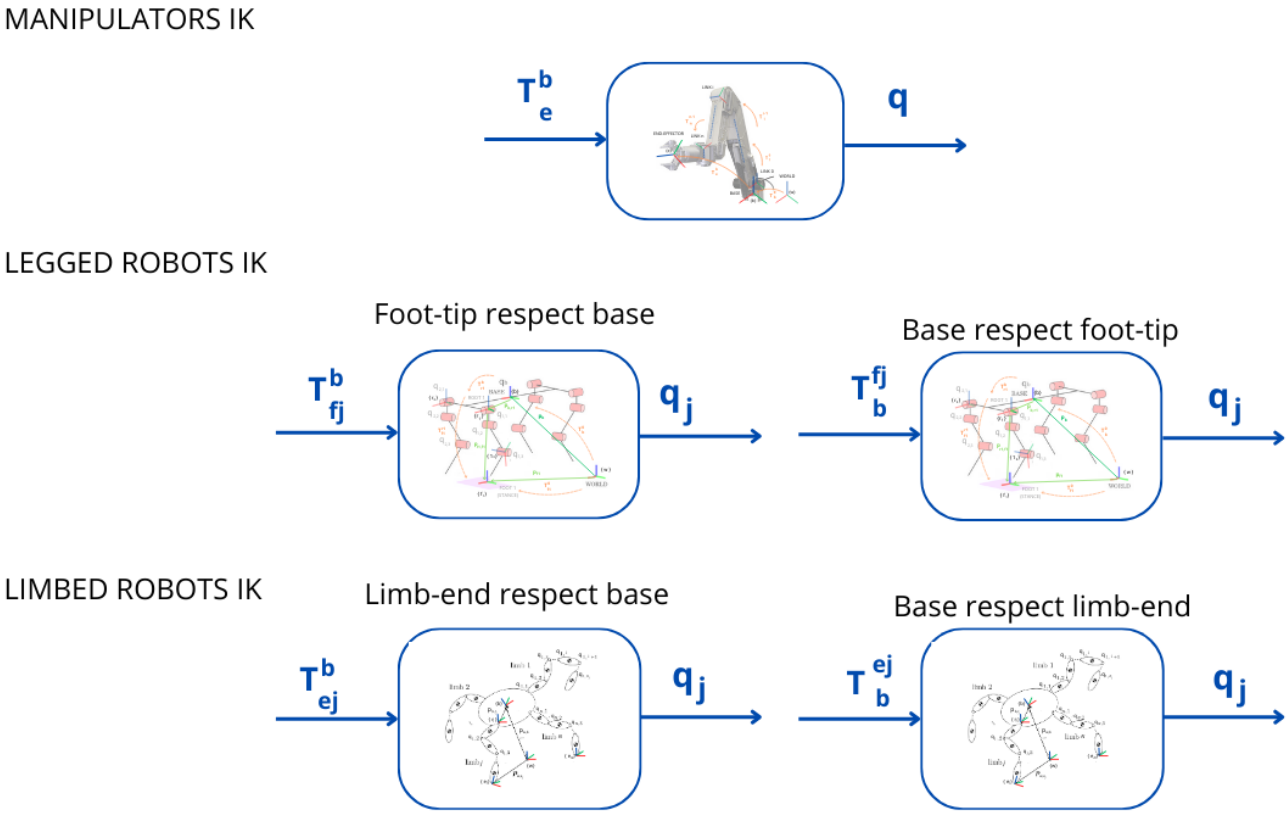



Figure 2.19: Scheme of FK problem for different robots

While FK has always one (and only one) solution, IK can have one, multiple (even infinite), or no solutions.

From the definition of workspace 2.1.7, it is clear that if a pose outside of it is chosen, IK cannot be solved. Due to mechanical constraints, link collisions, or design choice, the joint space $\mathbf{q} \in \mathbb{R}^n$ is an ideal space. The counter-image of the workspace is a subspace of the joint space, this can be defined as constrained joint space $\bar{\mathbf{q}} \subseteq \mathbf{q}$. For example, for a single revolute joint mechanism, the joint space is $\mathbf{q} \in \mathbb{R}$, but if it can move only in between $-\pi$, $\pi$, then the constrained joint space is $\bar{\mathbf{q}} \in [-\pi, \pi]$.

There isn't a standard way to solve IK, some techniques are:

- **FK algebraic inversion**:
  It consists of deriving the FK solution and inverting it. But, whenever revolute joints are involved, it is clear that trigonometric functions of the joint variables appear in FK, not invertible in $\bar{\mathbf{q}}$. Then, to invert it, assumptions on the constrained joint space domain are needed. Depending on the domain restriction, different solutions are found. Even when a solution exists, finding a closed-form IK solution is hard and sometimes not possible. Having a closed-form solution makes computation faster and more reliable, expressed as:

  $$\mathbf{q} = f_{IK}(\mathbf{x_f}) \tag{2.10}$$

  Where $\mathbf{x_f}$ denotes the final frame task space vector. When it is not possible, numerical methods are used.

- **Numerical methods**:
  To solve non-linear equations, numerical methods such as Newton-Raphson[90] are often used in robotics.

- **Inverse differential kinematics**:
  This solution depends on the differential kinematic formulation, which will be discussed in section2.2.3. For IK solution, once an explicit velocity relation is established between joint space and final frame velocity (goal of DK), it is inverted and integrated.

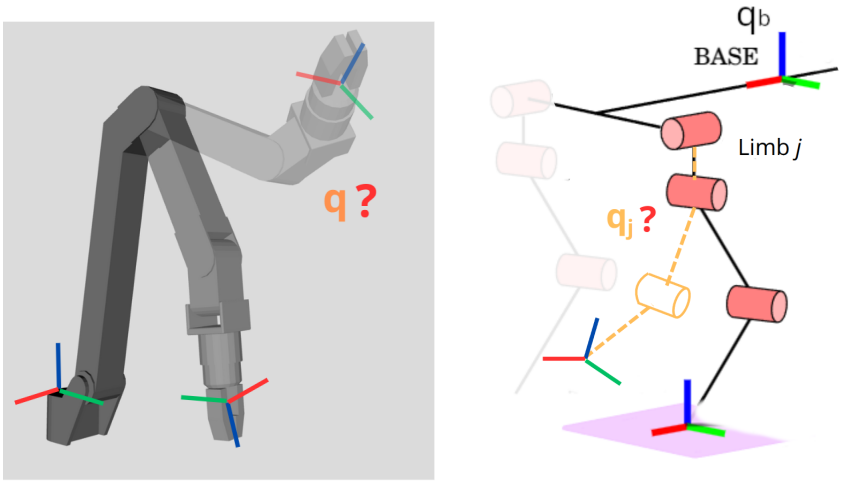
Next, an overview of the IK problem for the robotics structure of interest is done. However, due to the absence of a standard method, the details of the resolution are not addressed here.

## Manipulators and Single-Leg/Limb

The problem of IK for manipulators has already been introduced, formally:
*Given a desired end-effector pose* $\mathbf{T}_{\mathbf{e}}^{\mathbf{0}}$ *(*$\mathbf{T}_{\mathbf{e}}^{\mathbf{w}}$*),or as* $\mathbf{x}$*, derive the joint configuration* $\mathbf{q}$ *to reach it.*

Again, as for FK, the same approaches can be used for single legs/limbs modeled from the base root attachment to the tip/end. In that case, the problem is formulated as follows:
*Given a desired limb j-end/leg j-tip pose* $\mathbf{T}_{\mathbf{f_j/e_j}}^{\mathbf{b}}$ *(*$\mathbf{T}_{\mathbf{f_j/e_j}}^{\mathbf{w}}$*),or as* $\mathbf{x}_{\mathbf{f_j/e_j}}$*, derive the joint configuration* $\mathbf{q_j}$ *to reach it.*

The IK problems described above are represented in Fig.2.20.




(a) (b)

Figure 2.20: IK problem for Manipulators (a) and single leg/limb (b)

The resolution of the IK described above is needed to plan any arm/limb manipulation or single leg/limb step. A different approach is used when positioning a legged/limb robot base. The definition of this IK problem is described in the next part.

## Legged/Limbed Robots Base

The representation in Figure 2.19 of IK for legged/limbed robots base is not complete. In fact, when dealing with base motion, we can reformulate the problem as:
*The inverse kinematics problem consists of the determination of* ***all*** *legs/limbs joint variables corresponding to a given base position and orientation. The solution to this problem is of fundamental importance to transform the motion specifications, assigned to the base*

*frame, into the corresponding joint space motions that allow execution of the desired motion.*

The relation to derive is not the single leg/limb mapping from $\mathbf{T_w^{f_j/e_j}}$ to $\mathbf{q_j}$. To position the base, a **closed chain** structure (as in Fig.2.4b) is considered. Only legs/limbs in contact with the ground ($j \in J_c$) affect base positioning.

Formally: *Given a desired base pose* $\mathbf{T_b^{f_j/e_j}}$ *(from one of the contact legs/limbs* $j \in J_c$ *frames), derive all contact legs/limbs joint configurations* $\mathbf{q_c} \subseteq \mathbf{q}$ *to reach it.*

Here, two approaches are described, in both cases, rigid contact with the ground is assumed for the kinematic modeling. The first does not directly consider the closed chain, instead considers no penetration constraint of contact legs/limbs tip. The second instead relies on the parallelogram kinematic model to formalize and solve IK.

**Contact Leg/limb Motion Propagation without Penetration**

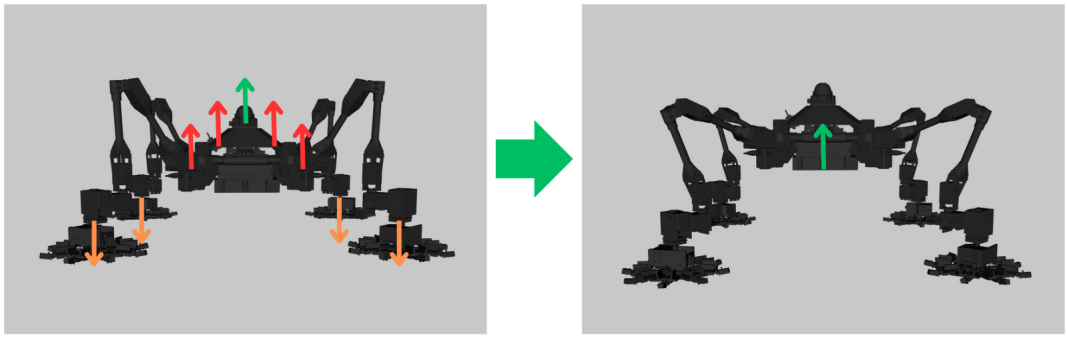
When the contact between leg/limb-tip and ground is rigid, if a motion occurs during contact, ground penetration is not possible. This causes a motion propagation towards the root, rigidly attached to the base. CConsidering only translational motion of the limb/leg in contact $j \in J_c$, if motion along 3D vector $\mathbf{v_{e_j/f_j}} = \begin{bmatrix} \mathbf{x_{e_j/f_j}} & \mathbf{y_{e_j/f_j}} & \mathbf{z_{e_j/f_j}} \end{bmatrix}^T$ this is mapped into a limb/leg root motion $\mathbf{v_{r_j}} = -\mathbf{v_{e_j/f_j}}$. According to rigid body theory [3], the motion is translational when all points move along parallel directions, otherwise it is rotational. Therefore, when all contact legs/limbs roots translate along the same direction, that motion is propagated to the robot base. For a base rotation, different root translations are combined.

In simple terms, positioning the base becomes a problem of contact legs/limbs position, using the same single root-to-tip model described before.

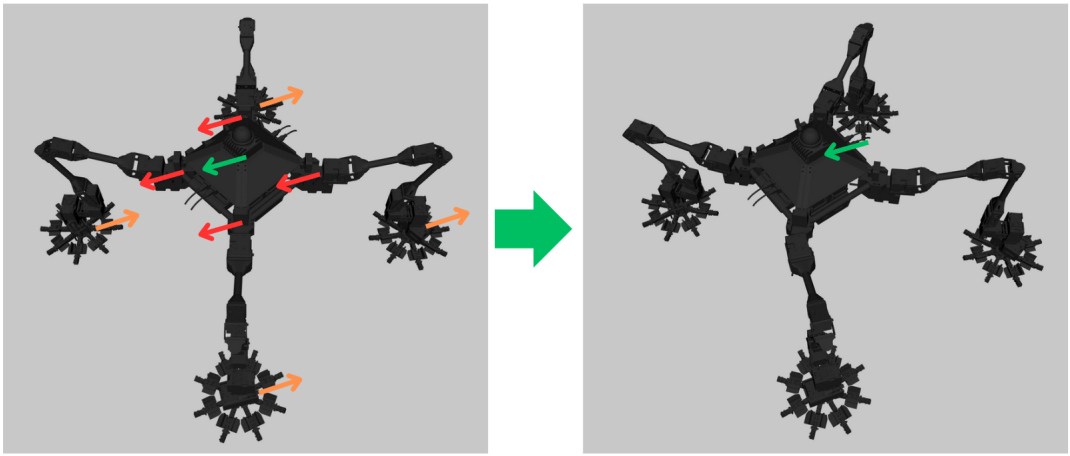
This approach is formulated in terms of "motion" and "changes in position", which is a topic of differential kinematics (treated in the next section). But it remains an IK problem if interpreted with the following steps:

1. Select a desired base pose $\mathbf{T_b^{f_j/e_j}}$ to $\mathbf{q_j}$.
2. Convert that pose as the configuration of contact legs $\mathbf{T^b_{f_j/e_j}}, \forall j \in J_c$.
3. Solve leg/limb IK $\forall j \in J_c$.

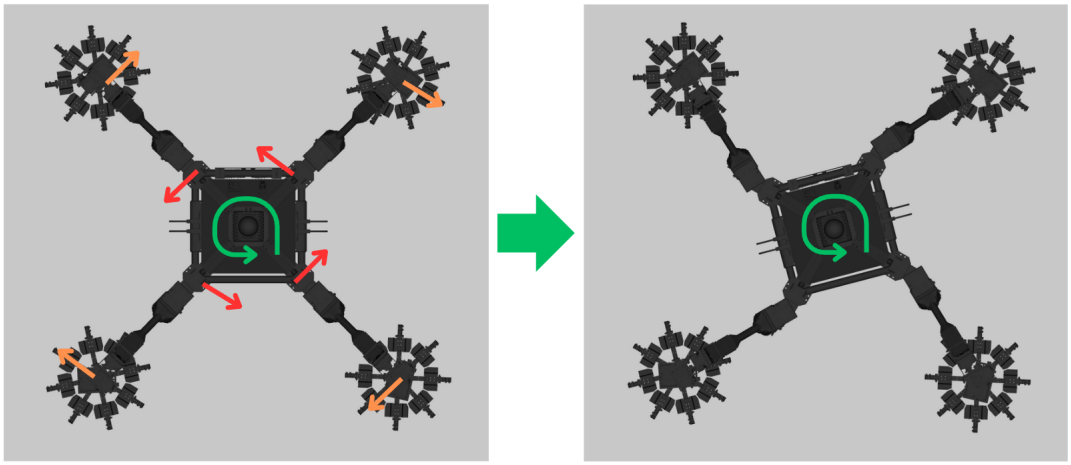
Figure 2.21 represents some examples of legs/limbs to base motion conversion.



(a) Z translation.



(b) XY translation.

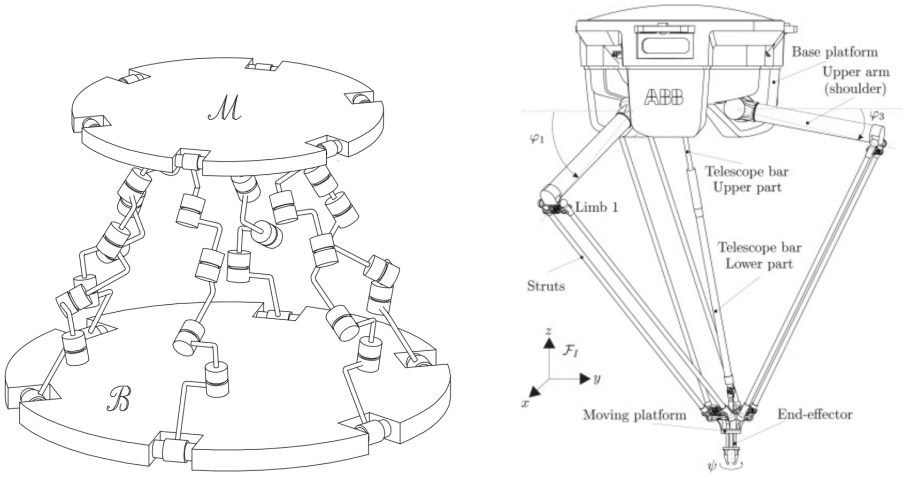


(c) Yaw rotation.

Figure 2.21: Different base motion as limb motion task. Orange arrows are objective limb position, reds are the new corresponding root pose, and green is the change on the base pose

### Parallel Manipulator Model

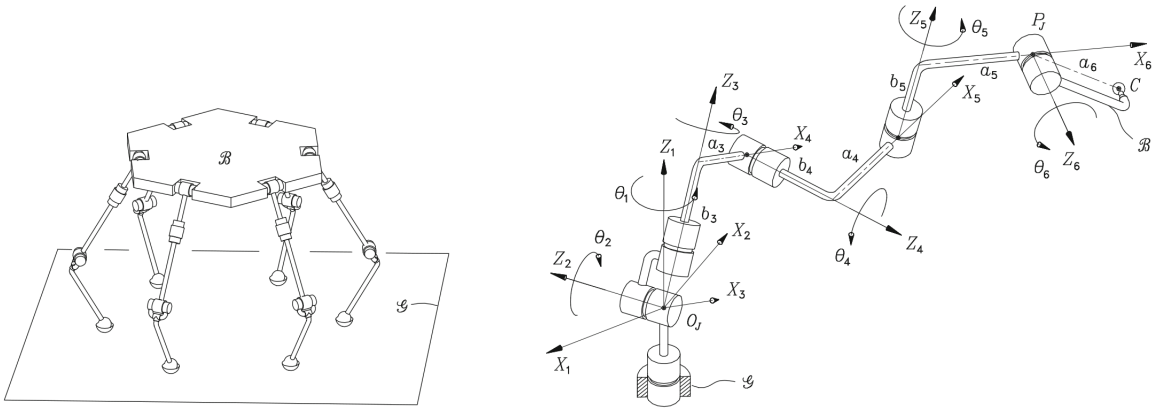
Unlike serial manipulators (open chain structures as Fig.2.1b), their parallel counterparts are composed of kinematic chains with closed subchains (as in Fig.2.22 below). Two frames can be distinguished, one fixed to the ground,$B$, and one capable of moving arbitrarily within its workspace, $M$.




(a) [3]Parallel manipulator scheme. (b) [55]ABB IRB.

Figure 2.22: Parallel manipulator general scheme and example

We can extend the parallel manipulator model[3] and consider the ground frame $G$ as fixed base (previously defined as $B$), and the robot moving base $B$ as a moving frame (previously defined as $M$) as in Fig.2.23. Each leg is modeled from the contact point to the base root with the classical manipulator model described before.




(a) Robot model as parallel manipulator. (b) Limb kinematic model.

Figure 2.23: [3] Closed Kinematic chain base motion

When modeling legged robot base kinematics with the parallelogram structure, legs in contact with the ground are considered. Using the same leg kinematic model as in section 2.2.1, The spherical joint of each leg j $q_{j,1}, q_{j,2}, q_{j,3}$ accounts for the coupling of the leg with the ground. This is just for modeling purposes, there is no real actuation on those joints. So, we can reduce the space of actuated joint variables to $\mathbf{q_a} \in \mathbb{R}^{n-3N}$, removing all spherical coordinates. A maneuver of $B$, given by a prescribed pose, can be achieved by suitable values of the contact limbs actuated-joints $\mathbf{q_a}$ variables, which thus leads to a problem of parallel-manipulator inverse kinematics[3].

### 2.2.3. Differential Kinematic

FK and IK have been defined as the direct and inverse mapping between joint space and task space. Such relations capture the robot configuration at a specific time instant. Differential kinematics (DK) instead derives the relationship between joint velocities and final frame linear $\dot{\mathbf{p}}_\mathbf{f}$ and angular $\omega_\mathbf{f}$ velocity[134]. Denoting the joint space as $\dot{\mathbf{q}} \in \mathbb{R}^n$, and the final frame velocity vector $\mathbf{v_f} = \begin{bmatrix} \dot{\mathbf{p}}_\mathbf{f} & \omega_\mathbf{f} \end{bmatrix}^T$, the mapping is described by:

$$\mathbf{v_f} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}} \tag{2.11}$$

$\mathbf{J}(\mathbf{q})$ is a $6 \times n$ matrix, called **geometrical Jacobian**. This matrix relation can be decomposed into a linear and angular part:

$$\mathbf{v_f} = \begin{bmatrix} \dot{\mathbf{p}}_\mathbf{f} \\ \omega_\mathbf{f} \end{bmatrix} = \begin{bmatrix} \mathbf{J_P}(\mathbf{q}) \\ \mathbf{J_O}(\mathbf{q}) \end{bmatrix} \cdot \dot{\mathbf{q}} \tag{2.12}$$

As always, the reference frame is important. If the notation is expanded, the jacobian matrix relating velocity of frame $\mathbf{f}$ in frame $\mathbf{0}$ can be denoted as ${}^\mathbf{0}\mathbf{J_f}$. The expression above represents the velocity of the final frame $\mathbf{v_f}$ in a chosen reference frame $\mathbf{0}$ . If velocity has to be represented in frame $\mathbf{k}$, it is easy to do under the hypothesis that $\mathbf{R_0^k}$ is constant [134][27]. It holds:

$${}^\mathbf{k}\mathbf{v_f} = \begin{bmatrix} {}^\mathbf{k}\dot{\mathbf{p}}_\mathbf{f} \\ {}^\mathbf{k}\omega_\mathbf{f} \end{bmatrix} = \begin{bmatrix} \mathbf{R_0^k} & \mathbf{0_{3\times3}} \\ \mathbf{0_{3\times3}} & \mathbf{R_0^k} \end{bmatrix} \cdot \begin{bmatrix} {}^\mathbf{0}\dot{\mathbf{p}}_\mathbf{f} \\ {}^\mathbf{0}\omega_\mathbf{f} \end{bmatrix} \tag{2.13}$$

Defining $\mathbf{J_0^k} = \begin{bmatrix} \mathbf{R_0^k} & \mathbf{0_{3\times3}} \\ \mathbf{0_{3\times3}} & \mathbf{R_0^k} \end{bmatrix}$, representing a transformation of velocity frames from $\mathbf{0}$ to

$\mathbf{k}$, from Eq.2.11, it is possible to derive the expression of ${}^{\mathbf{k}}\mathbf{v_f}$:

$$ {}^{\mathbf{k}}\mathbf{v_f} = \mathbf{J_0^k} \cdot \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}} \tag{2.14} $$

From now on, the velocity reference frame is neglected, as it depends on the robotic structure under analysis. It will be made explicit only if necessary.

When the final frame pose is represented in a minimal task space $\mathbf{x_f} \in \mathbb{R}^m$, it is possible to differentiate the forward kinematic relation, obtaining:

$$ \dot{\mathbf{x}}_\mathbf{f} = \mathbf{J_a}(\mathbf{q}) \cdot \dot{\mathbf{q}} \tag{2.15} $$

The **analytical Jacobian** $\mathbf{J_a}(\mathbf{q})$ is a $6 \times m$ matrix, and in general it is different from the geometric one. The Jacobian, not only establishes velocity relation, but the inversion and integration of that relation give us an algorithm to solve IK.

$$ \dot{\mathbf{q}} = \mathbf{J^{-1}}(\mathbf{q}) \cdot \mathbf{v_f} \tag{2.16} $$

and then:

$$ \mathbf{q}(t) = \int_0^t \dot{\mathbf{q}}(\varsigma) d\varsigma + \mathbf{q}(0) \tag{2.17} $$

Obviously, techniques to avoid drifting errors of the integrator are implemented, but it is out of this scope.

The importance of $\mathbf{J}(\mathbf{q})$ to be invertible is clear from Eq.2.16 . The values of $\mathbf{q}$ for which the Jacobian is not invertible ($det(\mathbf{J}(\mathbf{q})) = 0$, loss of rank) corresponds to a **singular configuration**. Singularities bring many problems, and so, the analysis of them is crucial for motion planning, and for system design.

The Jacobian matrix is also useful for understanding redundancy, an important concept in robotics, related to the relationship between joint space and task space[134][3][171]. A robot is redundant when the dimension of the task space $m$ is less than the joint space $n$, this brings additional freedom, but also complexity.

Another tool derived from the Jacobian matrix is the **Manipuilability Ellipsoid** theory, widely expanded and used in this research. This is a geometric representation of the manipulability theory first introduced by T.Yoshikawa[170], where a hyper-spherical joint-

space domain is mapped to a hyper-ellipsoidal task space, in the velocity or force:

$$\dot{\mathbf{q}}^T\dot{\mathbf{q}} = \mathbf{v_f}(\mathbf{J}\mathbf{J}^T)^{-1}\mathbf{v_f} = 1 \tag{2.18}$$

$$\tau^T\tau = {\gamma_\mathbf{e}}^T(\mathbf{J}\mathbf{J}^T)\gamma_\mathbf{e} = 1 \tag{2.19}$$

This concept is expanded in Section 3.3 for this research objective.

Finally, even if not related to kinematics, the Jacobian matrix provides a tool to solve the problem of **static**: *The goal of statics is to determine the relationship between the generalized forces applied to the end-effector and the generalized forces applied to the joints (forces for prismatic joints, torques for revolute joints) with the manipulator at an equilibrium configuration*[134].
The **kinetostatic duality** relationship can be proven with the principle of virtual work, deriving:

$$\tau = \mathbf{J^T}(\mathbf{q}) \cdot \gamma_\mathbf{e} \tag{2.20}$$

where $\tau \in \mathbb{R}^n$ is the vector of joint torques, and $\gamma_\mathbf{e} \in \mathbb{R}^m$ is the vector of final frame generalized forces (force and moments) in task space.

Again, there are some differences in the analysis of DK for manipulators and legged/limbed robots, which is explored in the next.

## Manipulators and Single-Leg/Limb

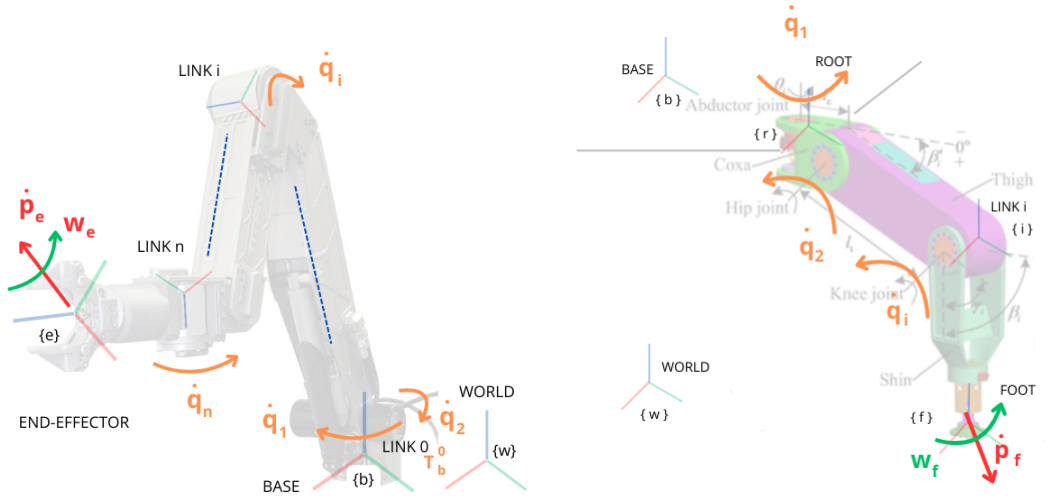
For serial link structures like a manipulator or a single leg, DK relation is defined as in Eq.2.11, with the same notation used for FK. Fig.2.24 represents an intuitive view of DK relation. If the frames are assigned using the DH convention, $\mathbf{J}(\mathbf{q})$ is derived column by column as:

$$\begin{bmatrix}\mathbf{j_{Pi}(q)}\\ \mathbf{j_{Oi}(q)}\end{bmatrix} = \begin{cases}\begin{bmatrix}\mathbf{z_{i-1}}\\ \mathbf{0}\end{bmatrix} & \text{if } i \text{ is prismatic}\\ \\ \begin{bmatrix}\mathbf{z_{i-1}(p_e - (p_{i-1})}\\ \mathbf{z_{i-1}}\end{bmatrix} & \text{if } i \text{ is revolute}\end{cases} \tag{2.21}$$

If the frames are not assigned with the DH convention, it is still possible to derive $\mathbf{J}(\mathbf{q})$ without this standard formulation. Furthermore, in terms of reference frame, Eq.2.21

provide the expression of ${}^{\mathbf{b}}\mathbf{J_e}(\mathbf{q})$ (velocity of end-effector in base frame) or ${}^{\mathbf{r}}\mathbf{J_f}(\mathbf{q})$ (velocity of foot/leg-tip in root frame), Eq.2.14 is used for a change of reference frame (for example, to move to world $\mathbf{w}$ frame or legged/limbed body frame $\mathbf{b}$).

The vectors in the statics equation 2.20 in this case are: all the arm or leg/limb joint torques $\tau \in \mathbb{R}^n$ and the generalized force applied at the end-effector or leg/limb-tip $\gamma_{\mathbf{e/f}} \in \mathbb{R}^m$. For example, in 3D space it can be $m = 6$ and $\gamma_{\mathbf{e/f}} = \begin{bmatrix} f_x & f_y & f_z & \mu_x & \mu_y & \mu_z \end{bmatrix}^T$




(a) DK of robotic arm.

(b) DK of root-foot robotic leg/limb.

Figure 2.24: Arm and Leg/Limb DK

## Legged/Limbed Robots Base

As discussed in the previous sections about FK and IK, the kinematics of N-legged/limbed robots involves the analysis of the base motion. With reference to Fig.2.25, the final goal is to derive a $6 \times n, (n = \sum_{j=1}^{N} n_j)$ Jacobian matrix $\mathbf{J}(\mathbf{q})$ representing the relationship between all legs/limbs joints velocity $\dot{\mathbf{q}} \in \mathbb{R}^n$ and the base velocity $\mathbf{v_b} = \begin{bmatrix} \dot{\mathbf{p}}_\mathbf{b} & \omega_\mathbf{b} \end{bmatrix}^T$.

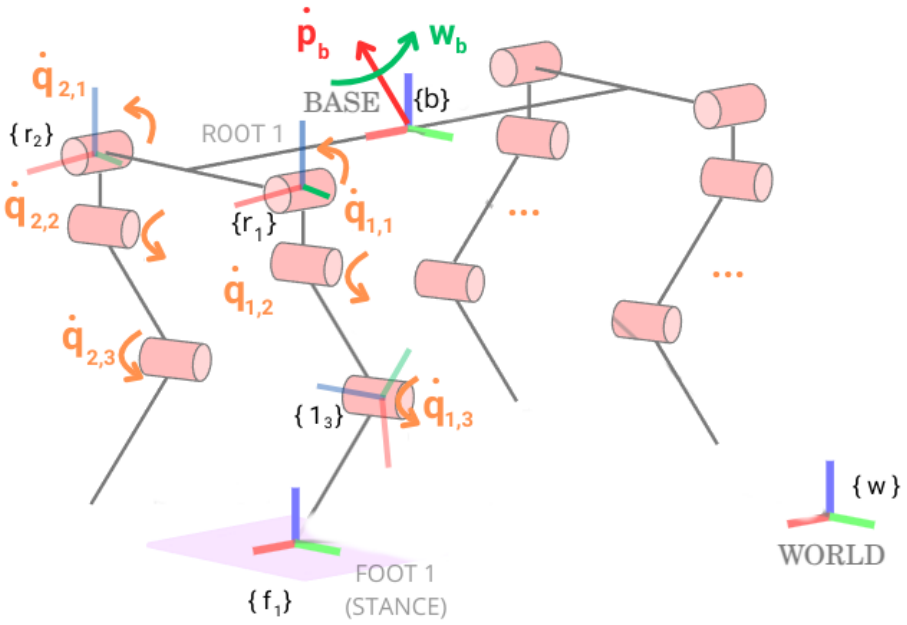



Figure 2.25: Legged robot DK relation

An alternative way to define the base velocity is $\dot{\mathbf{q}}_\mathbf{b}$ through the **floating base model**, described previously.

Different methods exist to derive this relationship, according to the kinematic model used. If the **parallel manipulator model** is used to describe the robot, theory on differential kinematics for closed chain structures exists [134][3]. To define the contact point-to-root kinematic model, one of the two approaches expressed previously is possible:

1. Extend the leg/limb model with a spherical joint and model the system inverting joint order (used for the parallel manipulator model).
2. use the contact leg/limb motion propagation without penetration approach, as defined in section2.2.2.

method (2) is employed in this thesis. From now on, the Jacobian matrix and velocity vectors' reference frame is important, so it is explicit. From the relation ${}^{e_j}\mathbf{v}_{r_j} = -{}^{r_j}\mathbf{v}_{e_j/f_j}$, it is just a matter of solving DK for the open chain root-to-tip leg/limb model (as in Eq.2.11):

$$ {}^{r_j}\mathbf{v}_{e_j/f_j} = {}^{r_j}\mathbf{J_j}(\mathbf{q_j}) \cdot \dot{\mathbf{q}}_\mathbf{j} $$

so, with the velocity relation assumed before:

$$ {}^{e_j/f_j}\mathbf{v}_{r_j} = -{}^{r_j}\mathbf{J_j}(\mathbf{q_j}) \cdot \dot{\mathbf{q}}_\mathbf{j} \tag{2.22} $$

The velocity vector ${}^{\mathbf{e_j/f_j}}\mathbf{v_{r_j}}$ is equivalent to the velocity of one of the cooperative manipulators' end-effector (corresponding to the limb root), measured in its base frame (the contact foot/limb-tip). This parallelism is represented in Fig.2.26

If we want to compute velocity of the limb root ${}^{\mathbf{b}}\mathbf{v_{r_j}}$, measured on the base frame:

$$ {}^{\mathbf{b}}\mathbf{v_{r_j}} = -\mathbf{J}^{\mathbf{b}\ \mathbf{r_j}}_{\mathbf{r_j}}\mathbf{J_j}(\mathbf{q_j}) \cdot \dot{\mathbf{q}}_\mathbf{j} = {}^{\mathbf{b}}\mathbf{J_j}(\mathbf{q_j}) \cdot \dot{\mathbf{q}}_\mathbf{j} \tag{2.23} $$

The ideas of **cooperative manipulators**[16][21][22] give a different point of view. Each leg/limb in contact with the ground $j \in J_c$ can be modeled from the contact point to the root as a separate "manipulator", cooperating to move the base frame. This is a novel approach for legged/limbed robots, introduced in this research, the details on this theory are addressed in in section 3.3 of the next chapter.

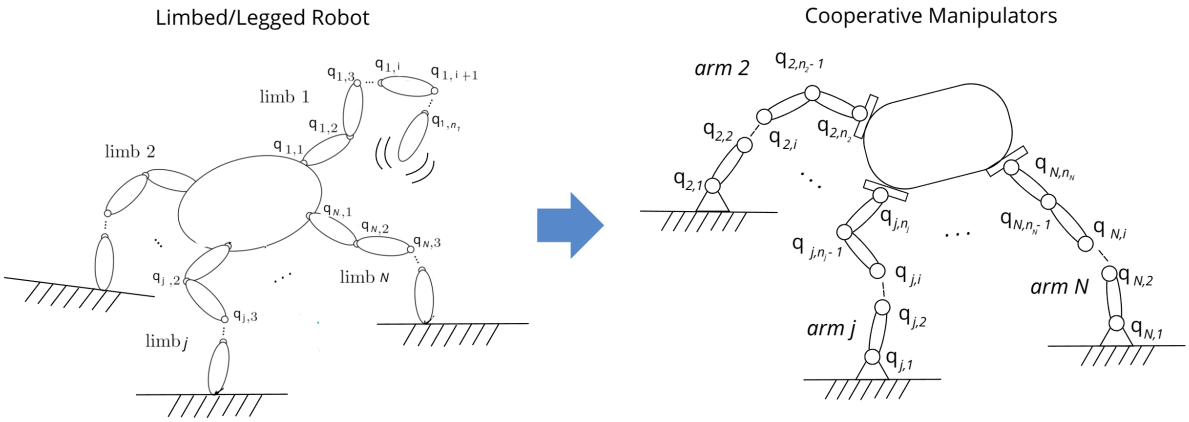



Figure 2.26: Parallelism between limbed/legged robot and cooperative manipulator

The final relationship derived from this approach is again in the form:

$$ \mathbf{v_b} = \mathbf{J}(\mathbf{q}) \cdot \dot{\mathbf{q}} \tag{2.24} $$

## 2.3. Dynamic Model

Dynamics complete the modeling tools needed to analyze and understand the motion of a mechanical system. While kinematics forget about the cause of motion, the objective of **dynamics modeling** is to derive the relationship between joint actuator torques and the motion of the structure[134]. The dynamics model is fundamental to derive the

systems' equations of motion (EoM), then used for **simulation** and **design of control algorithms**.

To derive those equations, two methods exist:

- **Lagrange formulation**: Systematic approach, based on energetic quantities differentiation. Defining the Lagrangian function as $\mathcal{L} = \mathcal{T} - \mathcal{U}$ (where $\mathcal{T}$ and $\mathcal{U}$ are respectively the kinematic and potential energy of the mechanical system) and $\xi$ the vector of generalized force associated to the joint variables $\mathbf{q}$:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}}\right)^T - \left(\frac{\partial \mathcal{L}}{\partial \mathbf{q}}\right)^T = \xi \tag{2.25}$$

- **Newton-Euler formulation**: based on a balance of forces and moments for rigid bodies. Recursive algorithms to solve dynamics are often based on this approach[134].

The Lagrange formulation is the method employed here because it derives the EoM in a compact analytical form, convenient for control design. Furthermore, dynamics branches into two problems:

- **Direct Dynamics**:
  *Given initial joint state and velocity* $\mathbf{q}(t_0)$, $\dot{\mathbf{q}}(t_0)$, *applied external generalized forces at final frame* $\mathbf{h_e}(t)$ *and joint torques* $\tau(t)$ *for* $t > t_0$, *determine joint acceleration* $\ddot{\mathbf{q}}(t)$ *for* $t > t_0$.
  Once this problem is solved, from $\ddot{\mathbf{q}}(t)$, it is possible to integrate and obtain $\dot{\mathbf{q}}(t)$, $\mathbf{q}(t)$, and then from kinematics, describe the motion of the frame of interest. This is the procedure used to simulate motion.

- **Inverse Dynamics**:
  *Given the desired joint motion* $\ddot{\mathbf{q}}(t)$, $\dot{\mathbf{q}}(t)$, $\mathbf{q}(t)$ *and knowing the external external generalized forces at final frame* $\mathbf{h_e}(t)$, *determine the joint torques* $\tau(t)$ *to achieve that motion.*
  Algorithms to control the system can be derived from the solution to this problem[20].

Next, a brief survey of the EoM derived from the dynamics of some robotic structures is presented.

### 2.3.1. Manipulators and Single-Leg/Limb

The theory on Manipulator robots is well-established, and a standard EoM in matrix form is derived with the Lagrange approach:

$$\mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F_v}\dot{\mathbf{q}} + \mathbf{F_s}sgn(\dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \tau - \mathbf{J}^T(\mathbf{q})\mathbf{h_e} \tag{2.26}$$

Where the matrices have the following meaning:

| | |
|---|---|
| $\mathbf{B}(\mathbf{q})$ | Inertia Matrix ($n \times n$, symmetric and positive definite). |
| $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ | Coriolis and Centrifugal Matrix ($n \times n$). |
| $\mathbf{F_v}$ | Viscous friction matrix ($n \times n$, diagonal). |
| $\mathbf{F_s}$ | Static friction matrix ($n \times n$, diagonal), coulomb friction model. |
| $\mathbf{g}(\mathbf{q})$ | Gravitational torque vector ($n \times 1$). |

The other symbols have the same meaning as discussed before. In particular, $\mathbf{J}^T(\mathbf{q})\mathbf{h_e}$ map the generalized force exerted by the EE to the joint space (similarly to the static relationship 2.20). When a generalized force $\mathbf{h_c}$ is applied at a generic point $\mathbf{C}$ in the structure, the mapping is straightforward, an additional contribution of $\tau_\mathbf{c} = \mathbf{J_c}^T(\mathbf{q})\mathbf{h_c}$ is subtracted at the left-hand side of the EoM [29].

The dynamic model above is highly coupled and configuration-dependent, for control purposes, a decoupled joint model can be derived.

#### Decoupled Model

The joint space vector $\mathbf{q}$ used until now, refers to the change of position of the link from the initial one, neglecting what happens at the motor level. Gearboxes are usually used for joint actuation, a gearbox provides a mechanical transmission between the motor and load, with reference to Fig.2.27, the following relationship holds:

$$\begin{cases} q_l = \frac{1}{N_{gear}} q_m \\ \dot{q}_l = \frac{1}{N_{gear}} \dot{q}_m \\ \tau_l = N_{gear} \tau_{lm} \end{cases} \tag{2.27}$$

Where $N_{gear}$ is the transmission ratio (or gear ratio), $\tau_l$ the torque at the load side (corresponding to $\tau_i$, $i^{th}$ joint torque of $\tau$), $\tau_{lm}$ the load torque transferred at motor side, $\tau_m$ the motor torque, $J_m$ the motor inertia, and $J_l$ the load inertia. Neglecting the friction

effect, the motor dynamics is defined by the following equation:

$$J_m \ddot{q}_m = \tau_m - \frac{\tau_l}{N_{gear}} \tag{2.28}$$

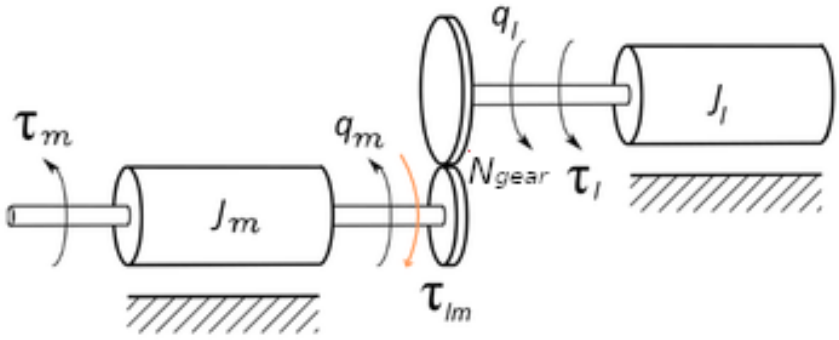



Figure 2.27: [134] Schematic representation of mechanical gear

Considering a single row of the dynamic equation of the manipulator (neglecting external forces and friction), the load torque for the $i^{th}$ motor-transmission-load system is:

$$\tau_i = \mathbf{B_i}^T(\mathbf{q})\ddot{\mathbf{q}} + \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{C_i}(\mathbf{q})\dot{\mathbf{q}} + g_i(\mathbf{q})$$

Then, collecting as residual the effect of Coriolis and centrifugal forces, gravitational load, and other joints ($j \not= i$) inertial effect:

$$\tau_{di} = \sum_{j \neq i} C_{ij}(\mathbf{q}) \cdot \dot{q}_j^2 + \sum_{j \neq i} B_{ij}(\mathbf{q}) \cdot \ddot{q}_j + g_i(\mathbf{q})$$

And approximating $B_{ii}(\mathbf{q})$, representing the effect of inertia of joint $i$ due to its acceleration, as $B_{ii}(\mathbf{q}) \simeq \overline{B_{ii}} = J_{li}$ (as an average value over frequent joint configurations), the equation for $i^{th}$ joint become:

$$\tau_i = J_{li}\ddot{q}_i + \tau_{di}$$

Introducing this load torque in Eq.2.28, the resulting decoupled model, represented in Fig.2.28 is:

$$\left(J_{mi} + \frac{J_{li}}{N_{gear,i}^2}\right)\ddot{q}_{mi} = \tau_{mi} - \frac{\tau_{di}}{N_{gear,i}} \quad i = 1, \ldots, n \tag{2.29}$$

It is important to notice that the residual term $\tau_{di}$ is scaled up by $N_{gear,i}$, meaning that,

high reduction ratios reduce the coupling effects, making the independent joint model a more accurate approximation.

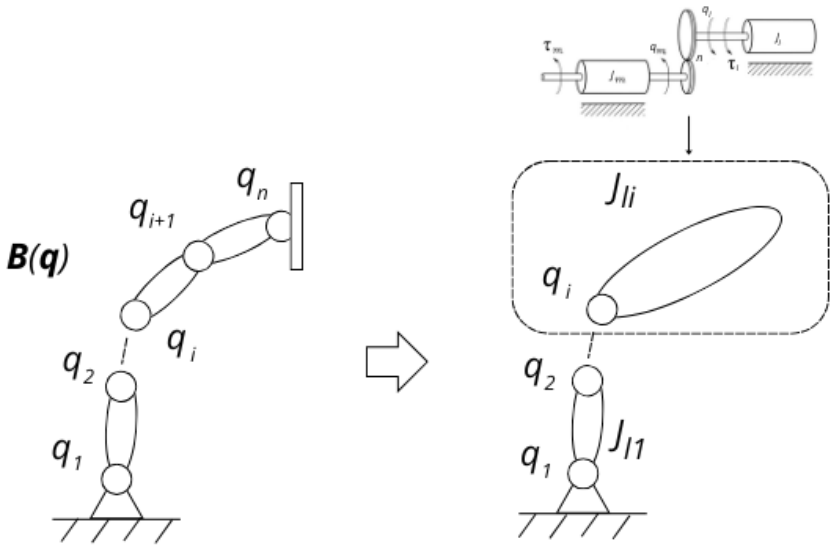



Figure 2.28: Schematic representation of decoupled manipulator model

Under the assumption that the base of a legged/limbed robot is static, the same equations describe a single leg/limb model from the root to the tip. In particular, this is useful in multipurpose multi-limb robots, when manipulation tasks are assigned, the EoM of the single limb can be considered as in Eq.2.26 or Eq.2.32.

### 2.3.2. Legged Robots, Walking Machines

The mechanical complexity of walking robots and their intrinsic closed kinematic chain has been commented on several times. For this reason, forces and moments propagate from one leg to another, further increasing coupling effects. Considering the different approaches for walking robots discussed before, if a "stance-foot" modeling approach is used, the parallel manipulator model can be used, where only contact legs are considered. The dynamic model of parallel manipulator [3] with a minimal DOF as all leg joints can be derived. With this modeling approach, changing the stance foot causes a switch in the dynamic model, making the dynamics model a state machine with a number of states exponential in the number of legs. If instead a non-minimal number of DOF is used and the floating base DOFs model is employed, ( previously defined as: $\mathbf{q} = \begin{bmatrix} \mathbf{q_b} & \mathbf{q_1} & \mathbf{q_2} & \dots & \mathbf{q_N} \end{bmatrix}^T$ ) it is possible to derive the EoM in a standard form with the Lagrange approach[56][72]

$\mathbf{q} \in \mathbb{R}^{n+6}, (n = \sum_j n_j)$:

$$\begin{cases} \mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{J_M}^T(\mathbf{q})\tau - \mathbf{J_C}^T(\mathbf{q})\mathbf{h_c} & \text{(2.30a)} \\ \mathbf{J_c}\ddot{\mathbf{q}} + \dot{\mathbf{J}}_\mathbf{c}\dot{\mathbf{q}} = 0 & \text{(2.30b)} \end{cases}$$

Eq.2.30b represents the holonomic contact constraint of null velocity at contact point $\mathbf{v_c} = \mathbf{J_c}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0}$, and the matrices involved include the self and coupling effects of all leg joints and base DOF, so they have bigger dimensions than Eq.2.26. The matrix $\mathbf{J_M}$ maps all leg joint torques in the space of generalized coordinates.

Under some assumptions, the dynamics can be simplified[132][110][72][162], providing a tool for the dynamical analysis of the robot, simple simulation, and controller design.

For example, as described in [162][132][161], the non-actuated (virtual DOF of the base) part of the EoM can be rewritten as the Newton-Euler equation of the whole system:

$$m(\ddot{\mathbf{c}} + \mathbf{g}) = \sum_i f_i \tag{2.31}$$

Where $\mathbf{c}$ is the position of the whole robot CoM, $\mathbf{g}$ is the vector of gravity, and $f_i$ are the external forces. Equation 2.31 doesn't provide details on the controlled DOFs, but it is important because it highlights that to generate any motion ($\ddot{c} \neq 0$) different from the gravitational falling effect, interaction forces $f_i$ are needed, so the goal of any motion algorithms will be to achieve a proper ground interaction. Other models include the cart-table and the inverted pendulum[150].

### 2.3.3. Limbed Robots, Multipurpose Machines

If modeling the dynamics of walking robots is complex, for multi-limbed multipurpose robots it is even harder. The main difference is in the interaction of each limb with the environment. In Eq.2.30a, the external force $\mathbf{h_c}$ model ground interaction during walking locomotion, only as pushing force, usually modeled using contact model as the friction cone [162][40]. For multi-limb robots instead, the interaction can include manipulation and so additional payload, climbing with pushing forces[120], and hybrid wheeled-legged locomotion[12]. An equation similar to 2.30a can be derived, but the modeling of $\mathbf{h_c}$ becomes harder.

## 2.4. Motion Control, State of the Art

The objective of motion control is to enable the system's autonomy, and two main approach are used in controlling robotics systems[165][178]:

- **Model Based**: the control algorithm exploits the knowledge of kinematics or dynamics model to compute the control action.
- **Model Free**: novel emerging approach like Reinforcement Learning[12][105], where the system learns a policy to execute tasks in the best way.

In robotics, model-based algorithms are intrinsically multi-layered, the problem can be generalized in the following steps:

1. Definition of the task (high-level)
2. Conversion of task into system DOFs (middle-level)
3. Actuation of DOFs (low-level)

Each level has more functions, and the accuracy and reliability of control can be ensured only when the system state is continuously observed, implementing a **feedback** control strategy[134][27].

This closed-loop control principle is defined as the "sense-plan-act" architecture (in Fig.2.29), composed of the main components to achieve autonomy:

- **sensors**: To perceive the internal state (proprioceptive) and the surroundings (exteroceptive)
- **actuators**: To effectively interact with the environment and perform actions.
- **controllers**: connect perception and action by a control logic, as the one defined above.

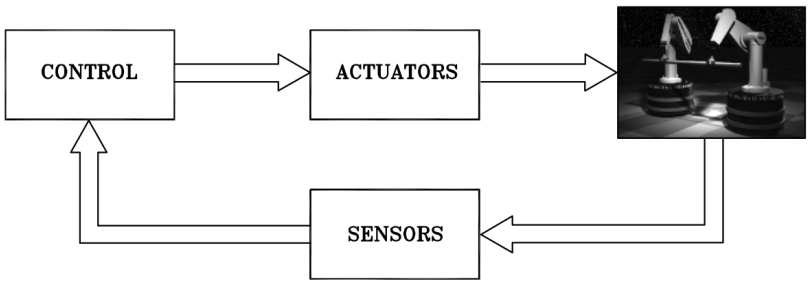



Figure 2.29: [134] Closed loop sense-plan-act architecture

Next, the control architecture for manipulators is defined, then, legged and multi-limbed robots are discussed.

### 2.4.1. Manipulators

In manipulator robots, the desired task is defined by a sequence of the EE positions, to be reached autonomously without colliding with obstacles. The main functions of the control architecture layer, represented in Fig.2.30, are:

- **High-Level**: Tasks are generated autonomously by intelligent decision algorithms or by an operator.
- **Middle-Level**: After generating the trajectory between the initial state (computed with FK from joint feedback) and the final state in EE coordinates, tasks are converted in joint space by IK. Then a control algorithm is used to compute the actuator input from the joint state reference.
- **Low-Level**: Closed loop control of actuator command, like the motor torque's proportional integral derivative (PID) feedback control.

Additionally, **state estimator** algorithms are included to keep track of the internal state of the system from sensor feedback.

The high-level task generator can be operated manually, or with more intelligent algorithms ensuring a complete level of autonomy. For example, in pick and place tasks or in machining, feedback from cameras and other sensors is used to generate the sequence of operations once the general task has been defined[27].

For the middle-level control layer, only the control in the joint space is described here, but working directly in the task space is possible with appropriate algorithms[134].

There are different choices for the control algorithms in the **axis controller**, which is responsible for moving the joints in the desired position, in the **model based** context, some of the most common are:

- **Indipendent Joint Control** (IJC): Decentralized control, based on the decoupled model defined in Eq.2.32.
- **Inverse Dynamics** (ID): From the desired joint state, dynamics equation 2.26 is solved in $\tau$. This approach requires an accurate estimation of dynamic model parameters, often hard to obtain.
- **Adaptive**: Based on adaptive control theory.
- **Model Predictive Control** (MPC): Find the best control input as an optimization problem using the system model.

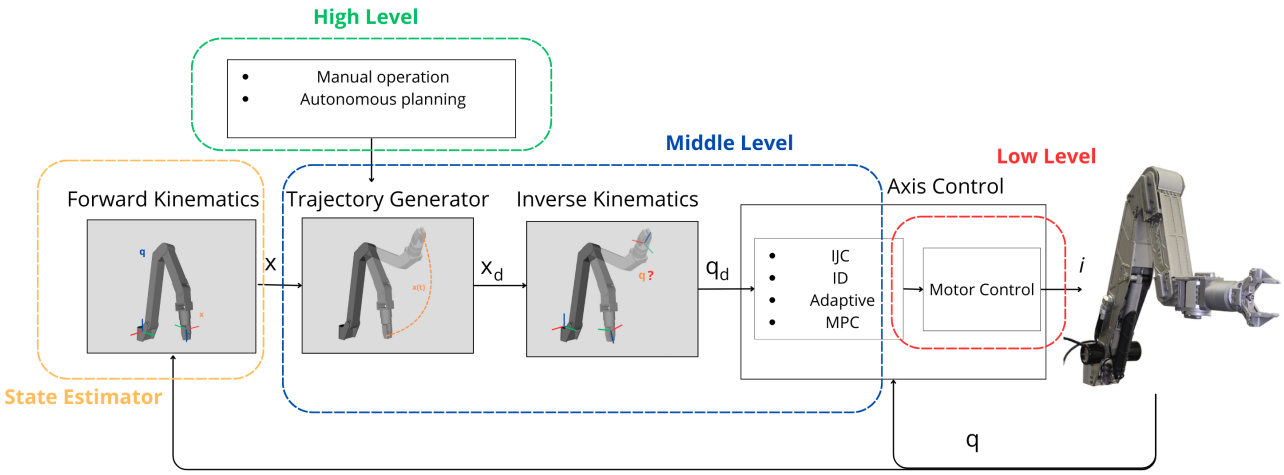



Figure 2.30: Schematic representation of manipulator control architecture

The IJC is a simple approach to controlling robot motion, for this reason, it is often used in robot prototypes.

## Indipendent Joint Controller

The decoupled joint dynamics described by Eq.2.32 is a good starting point for the design of a decentralized closed-loop control system of the whole robot.

In the Laplace domain, the transfer function from torque to position of each joint motor is:

$$G_i(s) = \frac{q_{mi}(s)}{\tau_{mi}(s)} = \frac{\frac{1}{J_{mi}+\frac{J_{li}}{n_i^2}}}{s^2} \quad i = 1, \ldots, n \tag{2.32}$$

The idea behind a decentralized control structure is to control each subsystem (joint motor) independently from the others, as represented in Fig.**??** below.

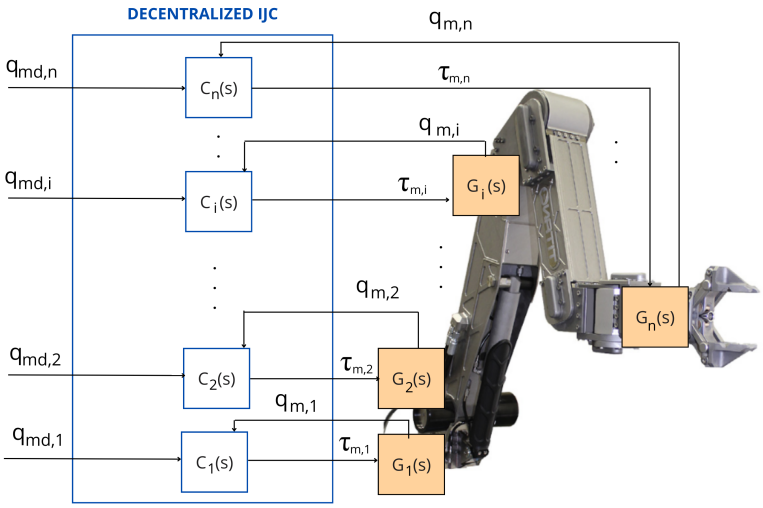



Figure 2.31: Schematic representation of a decentralized control structure for a manipulator

Fig.2.32 represents the structure of each controller. The motor joint position is often controlled using a cascade controller structure(P+PI), with a proportional(P) in position controller and a proportional-integral(PI) in velocity, and a feed-forward velocity compensation, the controllers are properly tuned accordingly to the system requirements[14].

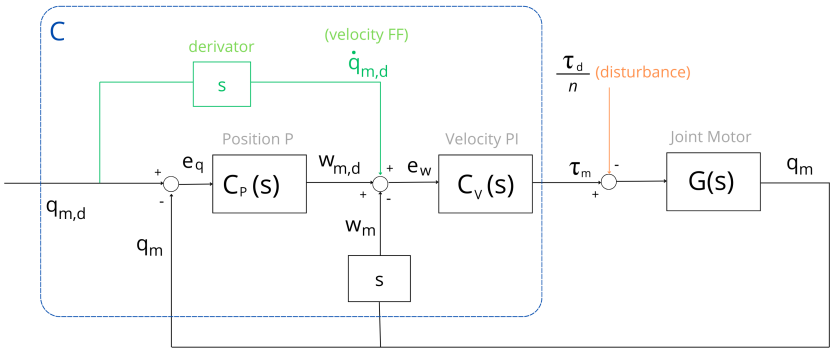



Figure 2.32: Block scheme of P+PI controller of each joint motor

Where the controllers have the following expressions in the Laplace domain:

$C_P(s) = k_{pp} \quad C_V(s) = k_{pv}\frac{1+sT_{iv}}{sT_{iv}}$

### 2.4.2. Legged Robots, Walking Machines

Legged robots introduce additional levels of complexity to the problem of motion control. A set of base positions in space defines the motion task, to be executed with a chosen gait without falling or colliding with obstacles. First of all, common for all mobile robots, the ability to move the base in the environment introduces a higher level of freedom, requiring a further ramification of control:

- **Global planning/control**: long-term decision component, works with low frequency to define the trajectory from an initial point to a destination[123]. This introduces the additional problem of understanding the environment through localization and mapping algorithms [49][12].
- **Local planning/control**: ensure the execution of the global plan, working with high frequency. In legged robots, some of its main components are: gait scheduler, body trajectory control, and leg control (these elements are discussed in the next).

In manipulators, the global planner is the high-level task generator, while the local planner is the architecture below that moves the robot.

Differently from wheeled robots, legged robots' motion is intrinsically discrete, with the advantage of higher traversability capabilities. But as introduced in chapter 1, legs bring advantages but additional problems, such as base stability. Details on the techniques to evaluate stable conditions for the base position are discussed in chapter 3. Intuitively, any legged system, including humans and animals, respects some configuration conditions

to avoid falling, due to gravity and other inertial effect. For this reason, during motion, legs are positioned appropriately to maintain stability, humans and animals do it from a learning process, but machines must be instructed to do it.

The definition of leg contact points along the trajectory is a complex task achieved by the global planner[143]. Leg order is decided by the **gait scheduler**, which generates a sequence of leg motion with a timing order, to move the base in a desired direction [150][67][115]. The **body trajectory controller** instead, keeps the body position in a stable configuration.

The higher-level components of this architecture are out of the focus of this research, instead, the different techniques to control leg motion from the base specification are discussed now.

- **Whole Body Controller** (WBC): Optimization-based control algorithm. Based on the desired base position and the dynamic model of the system, a constrained optimization problem is solved to compute the torque reference for all legs actuators [12][40][41].
- **Model Predictive Control** (MPC): Similarly to the WBC this is an optimization-based control algorithms, which introduce the concept of receding horizon in its "Predictive" feature.
- **Inverse Dynamic Torque-Based control** (IDC): Similarly to the ID control for manipulators, it is based on the ID problem for legged robots. From the Dynamic model inversion and the desired motion, the required joint torques are computed. When controlling motor torque, Impedance/Admittance control of legs is possible [134].
- **Inverse Kinematic Joint control** (IKJC): From the desired base motion, base IK is computed, obtaining each leg joint state. Then each leg is controlled with one of the techniques discussed for manipulator axis control (decentralized IJC, ID, Adaptive, MPC).

WBC, MPC, and IDC account for the whole robot dynamic model, ensuring better terrain adaptability, and performance, but dynamic model identification is usually hard. In IKJC methods instead, each leg is controlled singularly, this is easy to implement but lacks terrain adaptability and ignores the coupled system dynamics.

As for manipulators, this set of control problems requires a multi-layered control architecture, one possible control scheme is represented in fig.2.33 below, respecting the "sense-plan-act philosophy". Including estimators for the state of the body, each leg, and the surroundings (map and terrain).

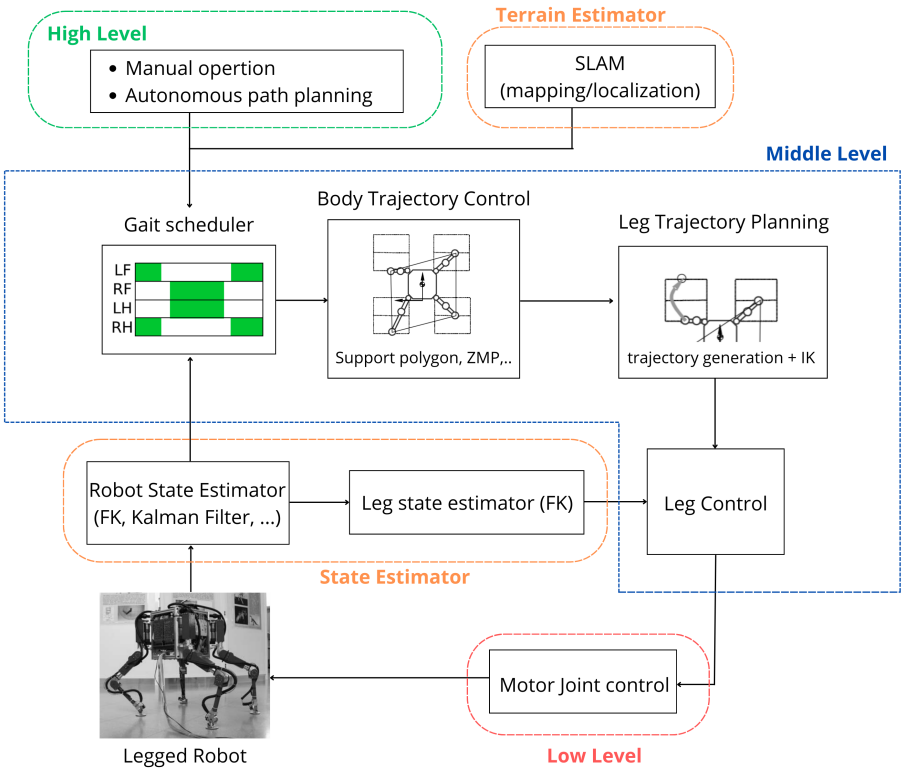



Figure 2.33: Schematic representation of one possible legged robot control architecture (some figures are taken from [56])

### 2.4.3. Limbed Robots, Multipurpose Machines

Defining a general control structure to control the motion of multi-limbed robots is not possible, due to their multipurpose nature. It is possible to give some task examples:

- **Climbing robots**: the motion task is defined by a set of base positions in space (as for walking robot), to be reached while maintaining wall/ceiling adhesion and without colliding with obstacles[154].
- **Walking/Climbing + Manipulating robots**: When limbs are designed also to manipulate objects in the environment. That means that tasks can be of two natures, reaching positions in space or moving limb tip in a sequence of positions, sometimes at the same time (walking while manipulating).
- **Driving + Walking robots**: legged-wheeled robots[12][57] add the additional legs DOF to the classical wheeled locomotion, making possible hybrid locomotion (like skating). The task is reaching poses in space, but additional decisional problems arise.
- **Hybrid robots**: For robots equipped with transformable EE at limb tips, the

complexity of modeling and control increases. Walking, climbing, driving, and manipulating tasks are unlocked.

While walking machines are constrained to stay on the ground, space multi-limbed robots move around gripping the surroundings [122][120][144]. Professor Oussama Khatib implemented a possible solution to this complex scenario, the "constrained consistent task oriented WBC"[74]. He defined a task-oriented control algorithm for humanoid robots, that can be generalized to different complex robotic structures. The details of this control algorithm are out of the scope of this research.

Another possible simpler solution is to consider again a multi-layered control structure, similar to the one discussed for walking machines (see Fig.2.33). In this case, additional complexity is included at a high level, where different types of tasks are generated. The output of high-level components will also include the type of task, and at middle-level decisions are taken accordingly. For the limb controller, reliable control algorithms (MPC, WBC, ID) are used to adapt to different tasks, such as moving the base, manipulating a heavy object, or gripping a rocky wall. Each module can be implemented using the concepts discussed for manipulators and climbing robots, but care must be taken due to the different tasks and situations addressed by the robot.